



MYRTUS

Multi-layer 360° dYnamic orchestration and interopeRable design environmenT for compute-continUum Systems

Deliverable:

D3.1- MYRTUS Reference Infrastructure v1

Due date of deliverable: (30-04-2025)

Actual submission date: (30-04-2025)

Start date of Project: 01 January 2024

Duration: 36 months

Responsible: Veena Rao(HIRO)

Dissemination level		
PU		x
DEC		
SEN		

Funded by the European Union, by grant No. 101135183. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.



DOCUMENT INFO

Author

Author	Company	E-mail
Maksimilian Mamiliaev	HIRO	maksimilian.mamiliaev@hiro-microdatacenters.nl
Veena Rao	HIRO	veena.rao@hiro-microdatacenters.nl
Rahul Sharma	HIRO	rahul.sharma@hiro-microdatacenters.nl
Alfonso Rodríguez	UPM	alfonso.rodriguezm@upm.es
Juan Encinas	UPM	juan.encinas@upm.es
Andres Otero	UPM	joseandres.otero@upm.es
Claudio Rubattu	UNISS	crubattu@uniss.it
Francesco Ratto	UNICA	francesco.ratto@unica.it
Francesca Palumbo	UNICA	francesca.palumbo@unica.it
Giulia Sedda	UNICA	giulia.sedda@unica.it
Danilo Pani	UNICA	danilo.pani@unica.it
Jessica Vacca	UNICA	jessica.vacca@unica.it
Tiziana Fanni	ABI	tiziana.fanni@abinsula.com
Giuseppe Meloni	ABI	giuseppe.meloni@abinsula.com
Fabio Marongiu	ABI	fabio.marongiu@abinsula.com
Katiuscia Zedda	ABI	katiuscia.zedda@abinula.com
Bart Driessen	TNO	bart.driessen@tno.nl
Giulia Biagioni	TNO	giulia.biagioni@tno.nl
Na Yan	KCL	na.2.yan@kcl.ac.uk
Yansha Deng	KCL	yansha.deng@kcl.ac.uk
Luca Castello	ARK	luca.castello@staff.aruba.it
Tristan Du Fretay	CRF	tristan.dufretay@crf.canon.fr
Francesco Regazzoni	USI	francesco.regazzoni@usi.ch
Subhadeep Banik	USI	subhadeep.banik@usi.ch

Document history

Document version #	Date	Change
0.0	07/03/2025	Definition of ToC and first draft of the document
0.1	12/03/2025	Contributions integrated
0.2	19/03/2025	Contributions integrated
0.3	26/03/2025	Contributions integrated
0.4	04/04/2025	Contributions integrated
0.5	10/04/2025	First complete release available
0.6	10/04/2025	Draft available for internal review
0.7	16/04/2025	Internal reviews received
0.8	25/04/2025	Internal reviews addressed.
0.9	30/04/2025	Document ready for submission



Document data

Keywords	WP3 Technical Pillar1 , Reference Infrastructure Definition and Architecture, edge-fog-cloud continuum.
	WP3 Progress and Accomplishments
Editor Address data	Name: Veena Rao
	Partner: HIRO
	Address: Prinses Marijkekade 11, 2273 AB Voorburg, Netherlands
	Phone: +31624759803
	Email: veena.rao@hiro-microdatacenters.nl



Table of Contents

1 Executive Summary	6
2 Introduction	7
2.1 State of the Art and EU relevant initiatives on Computing Continuum	7
2.2 Pillar 1 in a Nutshell	10
2.3 WP3 technical work summary	11
2.4 Structure of the document	12
2.5 Related Documents	12
3 MYRTUS Reference Infrastructure	13
3.1 Pillar 1 High-Level Architecture	13
3.2 Examples of MYRTUS compliant computing continuum infrastructure	16
3.3 EU-CEI mapping	18
4 Orthogonal aspects	21
4.1 Network	21
4.2 Privacy and Security	21
4.3 Gaia - X	24
4.4 Distributed Knowledge Base	33
5 Implementation and Integration	36
5.1 Implementation	36
5.2 Integration	57
6 Conclusion	61
7 Glossary	65
8 References	65
8.1 MYRTUS papers	65
8.2 Other references	66
9 Appendix	69

Acronyms

Acronym	Description
CGR	Coarse-Grained Reconfigurable
CGRA	Coarse-Grain Reconfigurable Array
CNN	Convolutional Neural Network
CoAP	Constrained Application Protocol
DFG	Data-flow graph
DKB	Distributed Knowledge Base



FPGA	Field Programmable Gate Array
FMDC	Fog Micro Data Center
FU	Functional Unit
HMPSoC	Heterogeneous Multiprocessor Systems-on-Chip
HTTP	HyperText Transfer Protocol
IaaS	Infrastructure as a Service
K3s	Lightweight Kubernetes
K8s	Standard Kubernetes
KPI	Key Performance Indicator
MA	MIRTO Agent
MIRTO	Multi-layer 360° dynamic RunTime Orchestration.
MQTT	Message Queuing Telemetry Transport
NATS	Neural Autonomic Transport System
OBI	Open Bus Interface
OS	Operating System
PaaS	Platform as a Service
PE	Processing Element
RBAC	Role-based access control
RR	Resource Registry
SaaS	Software as a Service
SHACL	Shape Constraint Language
SoC	System on Chip
UI	User Interface



1 Executive Summary

This deliverable provides the results of the work that has been conducted throughout the period of the first 16 months of the MYRTUS project, describing the activities and the result that were part of WP3 - “MYRTUS Reference Infrastructure Release 1”.

The goal of this WP is to build MYRTUS Reference Infrastructure for MYRTUS compliant computing continuum systems capable of supporting ubiquitous data collection and decentralized, intelligent decision-making.

The **MYRTUS Reference Infrastructure** is built as a composable layered edge-fog-edge computing continuum, integrating heterogeneous, autonomous, federated and collaborative computing nodes. The infrastructure provides *seamless execution of complex computational workflows across a heterogeneous continuum* of connected computational resources.

The infrastructure will provide customizable, adaptive and efficient edge/fog nodes including adaptable computing devices, small, energy efficient micro data centres, and a modular gateway, to bridge edge and cloud.

The progress covered in this deliverable will be the outcome of two tasks, namely T3.1 “MYRTUS Computing Continuum Components” and T3.2 “MYRTUS Computing Continuum Infrastructure”. T3.1 started at M3 of the project, and T3.2 started in M7.

The work conducted within T3.1 focused on the development of individual components that constitute the MYRTUS Reference Infrastructure including sensors, actuators, computing platforms, and communication-oriented devices. The listed elements are separated into the following two groups, according to their purpose. The first one serves more as cross-cutting technologies with computing platforms and communication devices. In this first case the development of novel computing elements is envisioned. While the second group includes sensors and actuators only and addresses more domain-specific needs. T3.2 has been dedicated to the integration of the MYRTUS computing continuum reference infrastructure, ensuring interoperability with open cloud environments and alignment with existing technological ecosystems.

This deliverable, which content is presented in [Table 1-1](#), will present the progress of Pillar 1, which was preliminary defined in D1.1 “UC Needs, requirements and KPIs v1”.

Table 1-1: D3.1 description from the Grant Agreement.

D3.1 - MYRTUS Reference Infrastructure v1 - M16

Description of MYRTUS computing continuum reference infrastructure and its components including SoA analysis. Stand-alone components assessment as well as partial sub-systems integrations are discussed.

Task Involved:

T3.1(M3-M18) - MYRTUS Computing Continuum Components[UPM(L), UNISS, UNICA, ABI, HIRO, USI, CRF]

T3.2(M7-M18) - MYRTUS Computing Continuum Infrastructure [HIRO(L), TNO, ABI, KCL, UNISS, UNICA, UPM,USI, CRF]



2 Introduction

The deliverable focuses on two objectives: refining Pillar 1 architecture, trying to position it with respect to the state of the art, and describing all the work that has been done within the WP3 both in terms of individual technologies and in terms of integration of the continuum.

2.1 State of the Art and EU relevant initiatives on Computing Continuum

It is widely recognized that the evolution of digital services and the rapid growth of connected devices have introduced new challenges concerning latency, bandwidth efficiency, data privacy, and energy consumption. Traditional cloud computing models, which rely on centralized data centers, have increasingly struggled to meet the demanding performance requirements of emerging applications such as autonomous driving and real-time industrial monitoring. Additionally, these cloud computing models have difficulty addressing privacy concerns, e.g., as needed in the healthcare sector. However, edge devices typically do not have enough storage capacity and limited computing capabilities to handle historical analytics and can hardly scale.

To address these limitations, the concept of a computing continuum has emerged. This paradigm promotes a hierarchical and distributed computing infrastructure, where computation, storage, and networking resources are flexibly distributed across cloud data centers, fog nodes, edge devices, and often IoT endpoints (intended as leaf with sensing and actuating capabilities only). The continuum aims to enable seamless service orchestration while optimizing latency, energy efficiency, scalability, privacy, security, and trust. In this document, we will focus mainly on the infrastructure related aspects, since all the topics around orchestration are related to WP5 and will be discussed in MYRTUS D5.1 Architecture.

In the following, we analyze the three main components of the continuum, namely the cloud, the fog and the edge, additionally to the available infrastructures and initiatives for the continuum.

Cloud computing remains the backbone of large-scale data storage and high-performance computing services. An example is the Microsoft Azure IoT Hub [Azure-IoT-2025], which provides end-to-end connectivity between IoT devices and cloud-based analytics services. Other similar platforms are offered also by Amazon [Aws-IoT-2025] and Google [Google-IoT-2025], while in Europe, cloud initiatives such as Gaia-X [Gaia-X-2025] aim to provide a federated, open, and secure data infrastructure, supporting data protection and sovereignty requirements specific to the European Union (EU). In MYRTUS we intend to align with the Gaia-X initiative, as discussed in [Section 4.3](#).

Fog computing was introduced by Cisco as an intermediate layer between the cloud and the edge, bringing computing and storage resources closer to the data sources [Bonomi-2012]. Fog nodes are typically deployed at network gateways, base stations, or micro data centers. They serve to preprocess data, reducing the communication overhead of transmitting raw data, and support location-aware services. In 2018, the OpenFog Consortium and the IEEE Standards Association collaboratively defined the OpenFog Reference Architecture for fog computing, which was adopted as an official standard by the IEEE, resulting in the standard IEEE 1934 [IEEE-standard-2025]. At the fog level, FogBus2 [FogBus2] is a lightweight, blockchain-based framework that integrates fog, edge, and cloud computing, offering data



integrity, platform independence, and energy efficiency for low-power IoT devices, but it is still in the research phase and lacks widespread industrial adoption.

Edge computing model is indeed particularly relevant in scenarios with strict latency, bandwidth, or privacy constraints. The Eclipse Kura [Eclipse-kura-2025] is an example of an open-source IoT edge framework (in this case considering the IoT edge as sensing nodes with processing capabilities) designed to simplify the development, deployment, and management of IoT applications.

All the above mentioned solutions are fundamentally silo ones, struggling to meet the plethora of diverse and adapting requirements of dynamic scenarios.

A preliminary example of cloud-edge integration is proposed by Asad Javed et al. [Javed-2018] with a solution called CEFIoT, where the edge devices composed of a Raspberry Pi cluster are put together with a public cloud solution. Local processing at the edge was enabled. Moreover, Kubernetes was leveraged for fault-tolerant management and on-the-fly automatic reconfiguration of the processing pipeline to handle both hardware and network connectivity based failures. This solution fails to address heterogeneity and workload reconfiguration, and it was limited to fault-tolerance purposes only.

The Intel's IoT Reference Architecture [Intel-IoT-2023] connects edge devices to cloud platforms through an intelligent network layer, offering strong industrial adoption thanks to Intel's hardware and software optimization, as well as seamless cloud-native support for Google cloud services, while also integrating end-to-end security mechanisms at various levels present some drawbacks, including potential vendor lock-in due to its primary optimization for Intel processors and Google cloud, as well as limited customization compared to open-source or research-driven frameworks.

Despite not mentioning the concept of fog layer nor the computing continuum one, the architecture proposed by OpenStack [OpenStack-2025] brings back together cloud and edge rather than considering them as isolated silos. It highlights the need for efficient communication in distributed infrastructures and discusses the challenge of synchronising abstract state propagation at the core infrastructure to cope with discontinuous network links. Furthermore, it emphasizes the necessity for tools that can manage application life cycles across potentially unreliable connections.

[Table 2-1](#) summarises the comparison of the MYRTUS solution with respect to all the above solutions.

Table 2-1 : comparison of MYRTUS Reference Infrastructure with respect to other solutions.

Project	AI-Driven Orchestration	Fog Layer (Intermediate)	Hardware Acceleration	Security & Trust	Open-Source vs. Proprietary	Interoperability	Energy Efficiency
MYRTUS	✓ Yes (MIRTO AI)	✓ Strong (FMDs)	✓ HMPSocs, FPGAs,	✓ Strong, three levels enabled	✓ Research & Open	✓ Strong	✓ AI-optimized



			RISC-V CGRAs				
Intel IoT Ref. Arch. [Intel-2017]	✗ No	✓ Strong	✓ Optimized Intel processors	✓ Intel Security	✗ Intel & Google cloud	⚠ Intel & Google focused	✓ Hardware-level
OpenStack edge [OpenStack-2025]	✗ No	✓ OpenStack-based	✗ No	✓ OpenStack security	✓ Open	✓ OpenStack-based	✗ No
Google IoT Arch. [Google-IoT-2025]	⚠ Limited	✗ No	✓ TPU/ML models	✓ Google Security	✗ Google-owned	⚠ Google-dependent	⚠ Limited
CEIoT [Javed-2018]	⚠ Limited	✓ Strong	✗ No	✓ Research	✓ Open	✓ Research	✓ Research
FogBus2 [FogBus2-2025]	✗ No	✓ Strong	✗ No	✓ Blockchain	✓ Open	✓ Cross-platform	✓ Lightweight
Azure IoT Hub [Azure-IoT-2025]	⚠ Limited	⚠ Limited	⚠ Limited	✓ Azure Policies	✗ Proprietary	✓ Azure Integration	⚠ Limited
AWS IoT [Aws-IoT-2025]	⚠ Limited	✓ Edge Gateway	✓ Greengrass	✓ AWS Policies	✗ Proprietary	✓ AWS Integration	✓ IoT Core
Gaia-X [Gaia-X-2025]	✓ Federated AI	✓ Federation	✗ No	✓ Trust Framework	✓ Open	✓ High	✓ Efficiency Focus



Eclipse Kura [Eclipse-kura-2025]	⚠ Limited	✓ Gateway Support	✓ Edge Hardware	✓ Security Policies	✓ Open	✓ MQTT, OPC-UA	✓ IoT Gateway
-------------------------------------	-----------	-------------------	-----------------	---------------------	--------	----------------	---------------

The concept of a computing continuum started appearing in academic literature and EU strategic documents around 2017-2018, in the context of edge-fog-cloud integration. Since then, different communities such as HiPEAC [HiPEAC-2025] (High Performance, Edge And Cloud computing) and AIOTI [AIOTI-2025] (Alliance for Internet of Things Innovation) actively contributed to formalize terms and concepts behind it. More recently, the EU-CEI [EU-CEI-IoT-2025] (European Cloud, Edge and IoT Continuum Initiative) initiative put together a wide range of research projects, developers and suppliers, business users and potential adopters of this new technological paradigm to define a reference architecture for the continuum to be promoted as a standard. EU-CEI identified eight categories, the building blocks of a computing continuum infrastructure, representing the technical processes to operate applications along the continuum [EU-CEI-2023]. EU-CEI motivations and goals are compatible with MYRTUS technologies and objectives and as already explained in D1.1 “UC Needs, requirements and KPIs v1”, we made an effort to map our technologies to the EU-CEI building blocks to make a first step towards the standardization of the technologies that are being developed in the project. The achievements of WP3 with respect to this mapping, are presented below in [Section 3.3](#).

2.2 Pillar 1 in a Nutshell

MYRTUS Pillar one intends to put together a computing continuum reference infrastructure. Indeed, provided the described scenario, it is certainly challenging finding a balance between performance, scalability, energy-efficiency, low latency, security, trust, and privacy when it comes to processes spread across edge-fog-cloud and at the same time handling computing and storage requirements efficiently. All these aspects are taken into consideration within component and infrastructure development in WP3. Each layer has its own specific mission, but all the novel components under development are implementing security and privacy measurements and are conceived to be as energy efficient as possible, while offering high performance. Moreover, MYRTUS computing continuum reference infrastructure will monitor a plethora of diverse metrics (including performance and energy efficiency) to allow dynamic workload and resources management (see WP5).

Additionally, when dealing with the computing continuum, the management of the heterogeneity is challenging when it comes to edge devices and network providers. The MYRTUS computing continuum reference infrastructure intends to exploit virtualization mechanisms to allow seamless integration and workload deployment across heterogeneous components.

A clear motivation for advancing continuum infrastructures is the need to support demanding workloads at the edge. For instance, deployment of machine learning and deep learning models directly at the edge layer, close to data sources, is still a widely acknowledged



challenge for continuum infrastructures to enable real-time inference without relying on cloud backends [Xu-2021]. Indeed, throughout heterogeneity, and developing ad-hoc solutions for edge AI inference, MYRTUS intends to tackle this challenge.

Besides these general challenges, individual components address specific challenges that are described in their respective sections.

2.3 WP3 technical work summary

From the implementation point of view, WP3 is composed of two tasks T3.1 “MYRTUS Computing Continuum Components” and T3.2 “MYRTUS Computing Continuum Infrastructure” that are summarized hereafter.

- **T3.1 started at M3 and is being led by UPM.** The task focuses on the implementation of the core components of the MYRTUS Reference Infrastructure, which include a vast array of sensors, platforms and devices. These components, along with already available commercial computing nodes, are vital for enabling the computing continuum implemented within the project. The development of these components was led by the driver of prioritising flexibility, performance, energy awareness, and efficiency as they are essential for the MYRTUS continuum computing approach. Within this task the consortium has worked both on the development of edge and fog components. UNISS, UNICA, and UPM worked together in order to explore the benefits of integrating heterogeneous multiprocessor systems-on-chip (HMPSoCs) and RISC-V processors with extended reconfigurable capabilities at the edge. At the fog layer, HIRO and ABI focussed on two different platforms, with different computing and storing capabilities. HIRO fog-based micro data center (FMDC) has large computing and storage capabilities, combining multiple computing cores together. ABI SenseIQ multi-sensor gateway acts as a bridge between edge and cloud, enabling local decision-making and triggering subsystem reconfigurations. Components development has been guided, in terms of security and privacy, by the requirements set by USI, which envisioned a strategy based on three different security levels to ensure data remains protected throughout the continuum while allowing flexibility. In addition to computing nodes, domain-specific components were also developed within T3.1 to meet the needs of the project’s use cases. For the Healthcare use case, UNICA designed a custom smart garment capable of collecting electromyographic and biomechanical signals while providing vibrotactile feedback. Meanwhile, CRF explored reconfigurability for the surveillance cameras used as edge devices to support the Mobility use case, enhancing their adaptability to dynamic environments.
- **T3.2 started at M7 of the project and is led by HIRO.** This task focuses on the integration of the components developed in T3.1 into the MYRTUS computing continuum reference infrastructure and at the same time ensure compatibility with already existing ecosystems using Gaia-X standards. TNO leads the Gaia-X initiative ensuring the infrastructure is aligned with these standards. KCL is supporting the definition of the network architecture, while USI is responsible for supervising the integration of security and privacy mechanisms at the infrastructure level. All the component providers play a crucial role in this task to enable the seamless integration of their components within the computing continuum. This collaborative effort aims



to create an infrastructure that is both scalable and adaptable along with being capable of supporting various applications.

The focus remains on ensuring the individual components development and partial integration of the infrastructure captured in this deliverable and highlighting the achieved results and made progress so far in Pillar 1.

2.4 Structure of the document

This document is structured as follows:

- [Section 3](#) provides the overview of the architecture ensuring the general understanding of the concept that lies underneath, followed up by the more detailed explanation of the layers that are integrated into the solution along with the components that represent the corresponding levels.
- [Section 4](#) focuses on the orthogonal aspects of the reference infrastructure that go through the whole developed solution and provides transversal supporting functions.
- [Section 5](#) provides the description of the components implemented in MYRTUS, with a discussion on implementation of individual components and the preliminary integration activities. Please note that each individual component represents one of the project's results. Each component is discussed also against the project Technical Operational Objectives (T.OOs) related to Pillar 1 and to its Key Performance Indicators (KPIs), defined in the GA and detailed in D1.1.
- At the end, [Section 6](#) wraps up the whole deliverable carrying out the general overview and outcomes.

2.5 Related Documents

- **D1.1 – UC Needs, requirements and KPIs v1.** This document sets the foundation for WP3, where the initial draft of the computing continuum reference architecture has been reported along with requirements, technical specifications, and KPIs.
- **D5.1 - MIRTO Cognitive Engine v1.** In D3.1 we define the initial specification of the distributed knowledge base and resource registry, which provides the infrastructure to store and share information across the different continuum layers, enabling MIRTO's managers in WP5 (see D5.1) to access and update contextual knowledge for AI-driven high level orchestration.
- **D7.1 - MYRTUS DPE v1.** In D3.1 we define the hardware components that constitute some of the node flavors available in MYRTUS. In particular, the hardware-accelerated edge nodes (i.e., the heterogeneous HMPSoCs and the RISC-V processor with CGRA-based extensions) will require some of the tools, frameworks and mechanisms, developed for the third step of the MYRTUS DPE.



3 MYRTUS Reference Infrastructure

This section describes the refinement and finalisation on the generic architecture of MYRTUS Reference Infrastructure (see [Section 3.1](#)). The high-level architecture of the MYRTUS Reference Infrastructure outlines the integration of the edge, fog, and cloud layers to create a unified computing continuum, with an emphasis on the partial integration of all layers in WP3. Examples of possible realization of MYRTUS- compliant infrastructures are described in [Section 3.2](#).

As already started in D1.1 the reference infrastructure of the MYRTUS Pillar 1 is mapped to the EU-CEI reference architecture, with most of its building blocks implemented and supported across all infrastructure layers. An updated description is provided in [Section 3.3](#).

3.1 Pillar 1 High-Level Architecture

The high-level overview of **MYRTUS Reference Infrastructure**, depicted in [Figure 3-1](#) has not changed with respect to the preliminary proposal in the Grant Agreement. It is built as a composable layered edge-fog-cloud computing continuum, integrating heterogeneous, autonomous, federated and collaborative computing components, summarizing for all layers: 1) the expected functionality, 2) involved resources, 3) distance from the data source, and 4) the expected processing latency. In the first 16 months of WP3 we have worked on refining the requirements and expected functionalities of each of these components, drafting the expected generic architecture (see [Figure 3-2](#)). The different interfaces involved for communication among components were identified and refined.

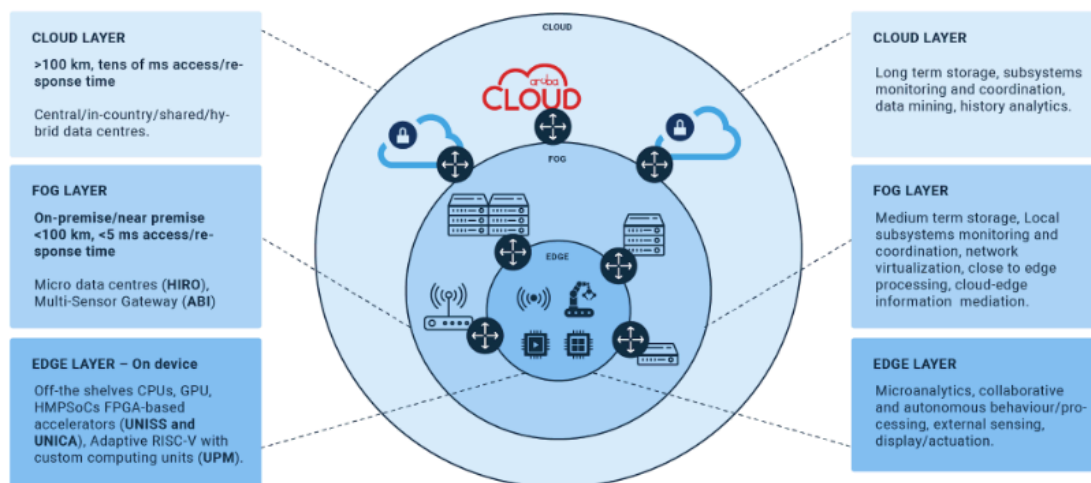


Figure 3-1: MYRTUS Reference Infrastructure.

The MYRTUS Reference Infrastructure is composed of three layers depicted in [Figure 3-2](#). Edge devices will be tightly coupled to fog nodes that are assumed to have, in turn, high availability connection to cloud. Fog layer balances data processing and storage tasks needed by edge devices by handling latency-sensitive operations that cannot be efficiently managed by the cloud Layer. As described above, the reference infrastructure provides *seamless execution of complex computational workflows across a heterogeneous continuum* of connected computational resources.



- **The edge layer** is close to sensors and actuators and performs local low-latency data processing. For MYRTUS, this layer consists of commercial off-the-shelf multicores, HMPSoCs, Field Programmable Gate Array (FPGA)-based reconfigurable accelerators and adaptive RISC-V processors with custom computing units providing complex computation including AI algorithm (see [Section 5](#)). This layer operates at the network's periphery, ensuring low-latency responses for time-critical applications. Positioned at the network edge, these devices are essential for fulfilling the low latency and energy efficiency requirements of contemporary workloads. The integration of FPGAs enhances their processing capabilities and adaptability, allowing them to efficiently handle resource-heavy tasks like real-time data analysis, machine learning inference, and sophisticated control operations. To optimize the orchestration of these tasks, containerization technologies are utilized, enabling the efficient deployment and management of workloads throughout the edge infrastructure. The combination of reconfigurable FPGA-based acceleration and container orchestration not only minimizes latency but also guarantees scalability, ease of deployment, and flexibility, fostering the creation of responsive, efficient, and scalable solutions within the MYRTUS project. In this regard, Kubernetes and Docker have been integrated to provide the containerized environments required to standardize deployments and optimize resource utilization. In particular, Lightweight Kubernetes (K3s) is employed to enhance performance efficiency and facilitate integration of the reconfigurable FPGA-based accelerators in a seamless manner for the user. The strategy consists of creating the container using Docker and managing it with K3s, delegating the container's execution to the native runtime.
- **The fog layer** acts as an intermediary between the cloud and edge layers, enabling real-time data processing while reducing latency and network congestion. In the edge-fog-cloud continuum, the fog layer is crucial for handling data that is too time-sensitive or resource-intensive to be sent to the cloud but requires more computational power than individual edge devices can provide. Devices operating in the fog layer rely on standard Kubernetes K8s for virtualization purposes, being thus fully aligned with the virtualization support implemented on the edge devices. They also offer enhanced processing and storage capabilities. Two main roles are envisioned for fog devices in the MYRTUS computing continuum: 1) to peer edges devices which are not visible to each other; 2) to collect, filter, and process data from edge devices, before sending only relevant or aggregated data to the cloud. This architecture is particularly beneficial for applications requiring real-time analytics, such as autonomous vehicles, industrial automation, and smart cities. Fog devices include FMDCs from HIRO and SenseIQ Multi-Sensor Smart Gateway from ABI (see [Section 5](#)).
- **The cloud layer**, being the most distant one from data sources, is used for long-term data storage and historical analytics. This layer is located, potentially, at a long distance from the point at which the data is generated. As a result, it has a relatively high response time. It represents the highest level of the architecture, responsible for high-capacity computing, large-scale data storage, and long-term data management. Within MYRTUS, we plan to leverage already existing public cloud infrastructures



supported by frameworks like the Gaia-X trust model along with orchestration tools such as Docker and standard Kubernetes K8s, based on the requirements of our specific use cases and their needs to maximise performance, providing scalability and ensuring security, privacy and trust. Integration of cloud with fog and edge layer is meant to form a computing continuum ensuring strong computing capacity in an energy-efficient manner.

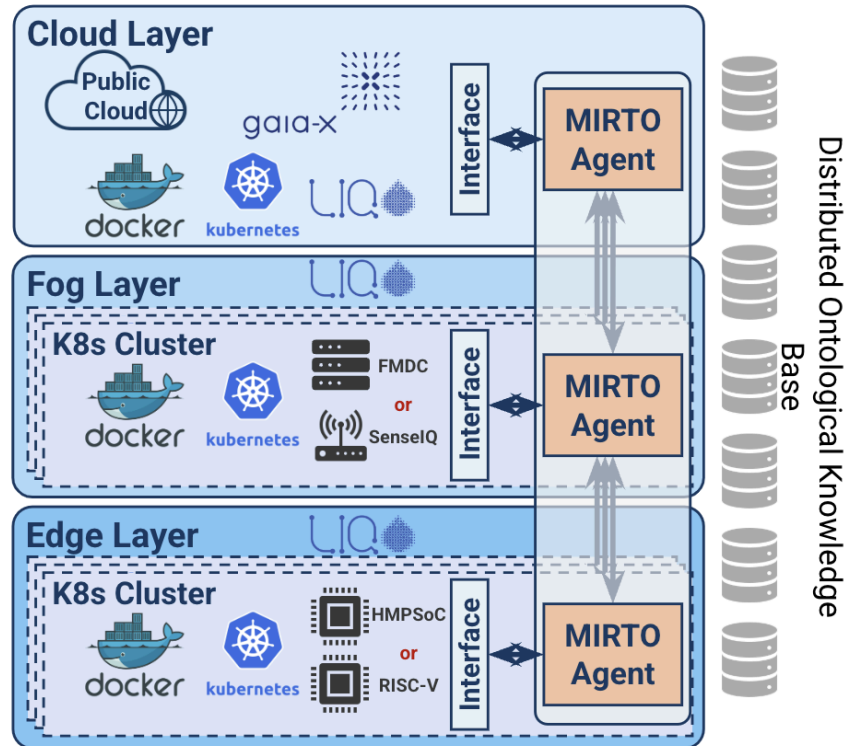


Figure 3-2: High level architecture.

The MYRTUS architecture in [Figure 3-2](#) uses **Kubernetes** as a low level orchestrator for managing and deploying applications across edge (K3s), fog (K8s) and cloud (K8s) infrastructure. This ensures a consistent way to organize workloads across different computing resources at each layer. For the workload deployments, all layers will have an interface to communicate with **MIRTO agents** (that are detailed in D5.1 and responsible for workload orchestration and resource management). To create a more seamless and unified environment, MYRTUS integrates Ligo [Ligo-2025], which enables dynamic connections and resource management across multiple Kubernetes clusters. **Ligo** helps to link cloud, fog, and edge deployments, making them work together in a seamless manner. This approach allows MIRTO agents to efficiently manage applications across unified infrastructure, reducing the complexity of handling distributed resources. By combining Kubernetes and Ligo, MYRTUS provides a flexible and scalable platform enabling continuum computing. To complement Kubernetes, which maintains low-level resource management within individual clusters, Ligo offers a way to view and connect resources across multiple Kubernetes deployments. Each layer will have its specific **interface** to communicate with the layer's specific MIRTO agent which, in turn, communicates with the other layers' MIRTO agent, ensuring efficient communication and task delegation and acts as high level workload orchestration.



From a logical perspective, all layers share a common set of concepts and definitions through a unified ontological knowledge base, even though this knowledge will be partitioned and managed differently across layers in the implementation. This way, data remains close to their consumers, but can be shared between layers for analysis and decision making. The **Distributed knowledge base (DKB)** provides a standard **interface** for storing and retrieving information, this helps all the components to access and store the information in a unified manner thereby enabling data sharing between all components present in different layers. A logical partition of the DKB is the *resource registry* that focuses on the accountability for distributing the most up-to-date state of the MYRTUS system. It ensures that the MIRTO agent has the real time data regarding the status of the hardware components and allocatable resource availability, which are essential for enabling optimal workload allocation.

In addition to the three layers that are presented in the above mentioned figure and involved in the continuum, the network is an integral part of the solution that allows seamless integration. The integration via network is implemented over the various protocols, including HTTP, MQTT, etc. Moreover, the reference infrastructure focuses also on supporting different security levels with varying cryptographic primitives. This set involves three main categories each presented with three different levels of security i.e Low, Medium and High. Those primitives are Encryption, Authentication, Key Exchange and Hashing. More details on Networking and Security aspects are covered in [Section 4](#).

3.2 Examples of MYRTUS compliant computing continuum infrastructure

In [Figure 3-3](#) and [Figure 3-4](#) two examples of realization of MYRTUS-compliant computing continuum infrastructures are shown, respectively depicting horizontal and vertical continuum realizations. In both cases, classic cloud and fog generic nodes are considered, while at the edge, exemplary, we are considering just FPGA nodes. Liqo takes care of “virtualizing” and securely connecting the resources exposed by a cluster (in green, called producer), and making them appear as if they were actually a (virtualized) node within a consumer cluster [Liqo-Peering-2025] (the K8s [Kubernetes-2025] Liqo node). Whether the resource sharing is vertical or horizontal is just a matter of implementation, dispensing on the scenario needs. In MYRTUS we will mainly adopt vertical continuum implementations, even if during this first year of the project we have also assessed the horizontal continuum realization (see [Section 5.2.1](#)).

Please note that more examples are provided in [Appendix 9](#), showing how it is possible to use Liqo to realize other types of horizontal (e.g. edge to edge) and vertical (fog peering different edges) node virtualization.

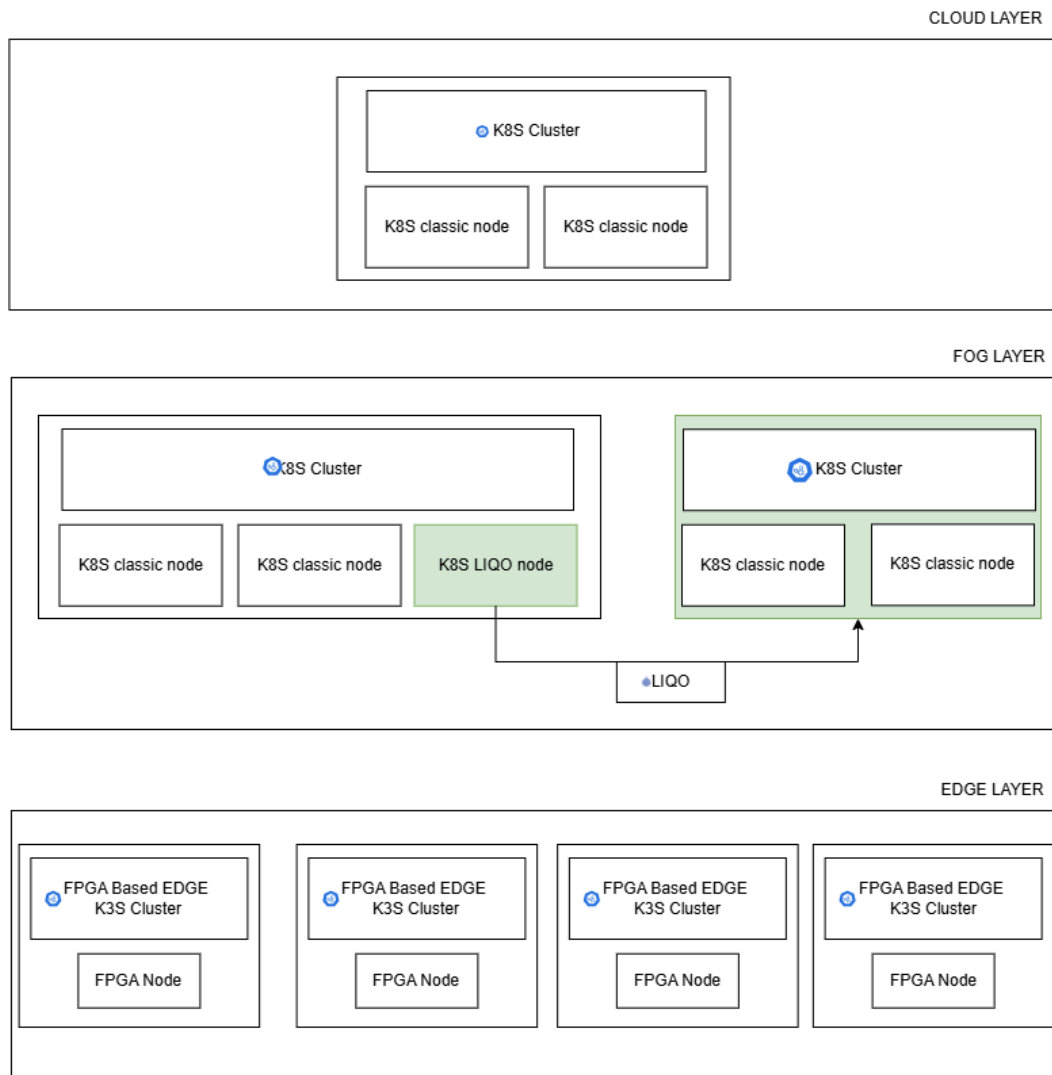


Figure 3-3: Horizontal (Fog to Fog) continuum

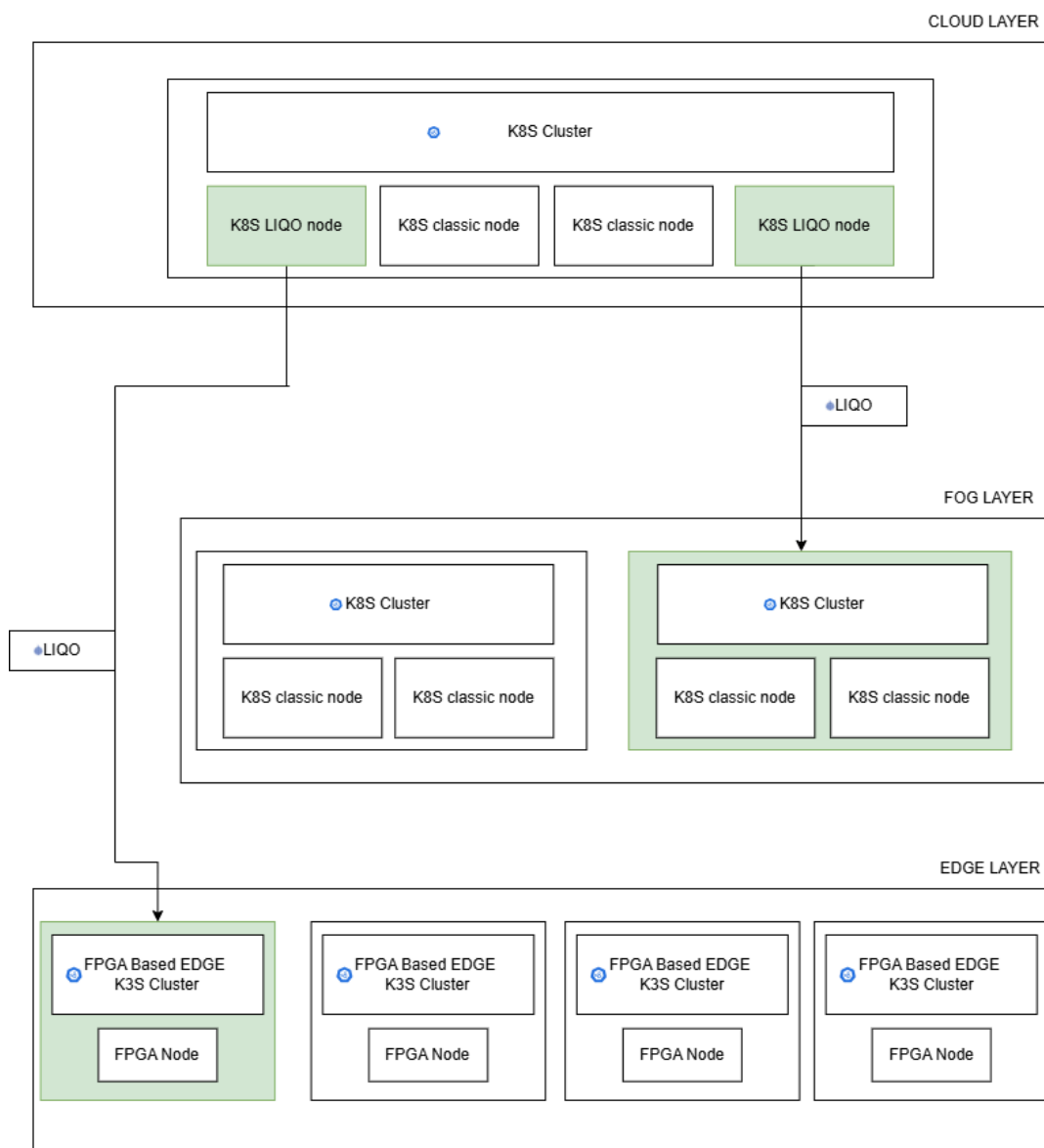


Figure 3-4: Vertical (Cloud-Fog-Edge) continuum.

3.3 EU-CEI mapping

The general approach of the MYRTUS Reference Infrastructure is closely aligned with the aims of the European Cloud, Edge and IoT Continuum Initiative (EU-CEI). The EU-CEI aims to define a common coordinated design and development strategy for cloud, edge and IoT continuum bringing together and promoting the cooperation between research projects, developers and suppliers, business users, and potential adopters of the continuum computing paradigm. One technical goal of the EU-CEI is to define a reference architecture for the continuum to be promoted as a standard and aligned with the solution architectures of projects operating in this domain. The MYRTUS concepts are mapped to the EU-CEI reference architecture defined in D1.1 “UC Needs, requirements and KPIs v1” document and further refined in [Palumbo-2025]. The EU-CEI identifies essential technical processes as fundamental building blocks for the continuum.



Figure 3-5 provides a more technical overview of the work done in the MYRTUS Reference Infrastructure when it comes to the EU-CEI mapping.

EU-CEI Building Blocks	MYRTUS Infrastructure		
Security & Privacy Trust & Reputation	3 security levels: High, Medium, Low	Compatibility with 	
Data Management	Processing Capabilities 	Data Storage 	
Resource Management & Orchestration	Low-Level Orchestration 	 	Embedded OS 
Network	Standard protocols and interfaces  		
Monitoring & Observability	Resource availability 	Power consumption 	Telemetry 

Figure 3-5: Mapping of MYRTUS infrastructure to the EU-CEI building blocks.

- **Security and Privacy** along with **Trust and Reputation** are fulfilled with the security primitives defined at each layer of the infrastructure with different security levels . In addition to it, the cloud layer meets the Gaia-X trust model requirements (see [Section 4](#)).
- **Data Management** is a distinct role of each layer in the MYRTUS Reference Infrastructure. The edge layer focuses on real-time processing close to data sources, ensuring low latency and responsiveness. The fog layer acts as a bridge, enabling localized processing and decision-making while managing communication between edge and cloud. The cloud layer supports large-scale data storage and long-term analytics, providing the computational power needed for heavy processing tasks. Consequently, all layers will have different storage and processing capabilities,
- **Resource Management** and **Orchestration** is achieved at the infrastructure level by integrating lightweight Kubernetes (K3s) [Lightweight-kubernetes-2025] and Docker, enabling efficient deployment and management of containerized workloads on edge devices. Local clusters are formed using K3s, while Liqo supports peering of remote K3s clusters to create a unified, multi-cluster infrastructure that allows seamless horizontal orchestration across geographically distributed edge nodes. K3s services will be enabled on the edge and fog layers by custom embedded Linux distributions.
- All the components of the MYRTUS Reference Infrastructure provide support for **Monitoring** and **Observability**. On the fog components, this is achieved by introducing the dedicated tools that ensure availability of real-time data regarding the actual state of the system (e.g., Prometheus which focuses on metrics and logs gathering, Grafana which supports current state visualisation and alerting). At the edge, on FPGA-based computing nodes, a custom monitoring infrastructure, designed to be compatible with all the MYRTUS developed components, is being developed in the project for gathering run-time power consumption and performance traces of the system operation.
- **Network** ensures the integration of all the components and is implemented via providing the various of the basic protocols such as HTTP, MQTT, etc. Along with that the FMDC also is capable of communication over Neural Autonomic Transport System (NATS), Constrained Application Protocol (CoAP) and others.



More technical details regarding the mapping to the EU-CEI building blocks are provided in [Section 5](#), where the preliminary implementation of the MYRTUS components is described.



4 Orthogonal aspects

4.1 Network

As part of T3.2, the communication architecture within the MYRTUS Reference Infrastructure has been defined. This involved, under KCL guidance, identifying typical communication scenarios across the edge-fog-cloud continuum and proposing corresponding architectural principles to guide implementation. Here are the identified communication-related requirements:

1. The MYRTUS Reference Infrastructure must include a network communication framework that ensures reliable, low-latency, and secure connectivity across the edge-fog-cloud layers. This framework should support heterogeneous networking technologies, including 5G, Wi-Fi 6, fiber-optic links, and SDN-based routing, to optimize performance across diverse deployment scenarios.
2. The communication-related components need to support monitoring of key network parameters, including bandwidth usage, latency, and channel state, to ensure efficient performance and reliability.
3. The MYRTUS Reference Infrastructure SHOULD support adaptive bandwidth allocation mechanisms to dynamically optimize uplink and downlink transmission rate based on workload demands and network conditions.

From the implementation point of view, edge components are expected to connect to the continuum with standard protocols (through Linux libraries). As an example, the HMPSoC accelerators from UNISS and UNICA are already capable of establishing secure connections via HTTP with the ABI gateway exchanging JSON packets. The gateway itself is extremely flexible in terms of connectivity interfaces, it is customizable with ad-hoc user defined interfaces, and already natively supports several protocols (e.g., HTTP, MQTT, etc.). The FMDC by HIRO is designed to seamlessly integrate with various edge components, leveraging standard protocols to maintain secure and efficient communication channels. It also supports a wide array of connectivity protocols such as HTTP, MQTT, CoAP, and more, providing flexibility and interoperability within the network.

4.2 Privacy and Security

The privacy and security in the MYRTUS Reference Infrastructure provides mechanisms for securing data and transactions between different computing nodes along with guaranteed components authentication.

To implement this, we have defined, for now, four main security primitives: Encryption, Authentication, Key Exchange and Hashing, each of them available at three different security levels (i.e., High, Medium and Low), as pointed out in [Table 4-1](#). The security level High is intended to cater for use cases where Post Quantum Secure algorithms are required by the underlying application that accesses these modules. One such example, could be encryption of video files captured by traffic cameras, that need to be preserved for future/long-term use. Since quantum computers may be realized in the near future, one must ensure that the encryption is post-quantum secure. On the other hand, the Low security level typically caters for lightweight applications where high throughput and low power/energy consumption is of more relevance to the application. For example, if encryption is needed for transmission of ephemeral data over an insecure channel, then throughput is a more relevant parameter to optimize and one can forgo the requirement that the encryption be post-quantum secure. In



the following subsections, we introduce how such modules may be constructed using docker containers.

Dockerized Modules: In order to implement these security primitives, each module along with the security levels it offers, are implemented as a Docker container. This has two advantages: having each module as an independent docker allows us to manage them more efficiently in a Kubernetes cluster infrastructure. And secondly, we have one docker for each module that handles the implementation level subtleties for all three security levels. This allows the underlying to just run each docker module with command line parameters specifying the exact security level and other performance characteristics like low power/energy etc.

Table 4-1: Security levels.

Security Primitive	High	Medium	Low
Encryption	Symmetric encryption primitives which are PQC resistant, e.g. AES-256	Symmetric encryption primitives which may not be PQC resistant but are suitable for current security threats, e.g. AES-128	Symmetric encryption primitives which may not be PQC resistant but are suitable for current security threats and consider computational complexity of the node, e.g. ASCON-128
Authentication	Digital signature schemes which are PQC resistant, following the NIST standard (CRYSTALS-Dilithium, FALCON, and SPHINCS+)	Digital signature schemes which may not be PQC resistant but are suitable for current security threats, e.g. DSA, RSA, ECDSA	Digital signature schemes which may not be PQC resistant but are suitable for current security threats and consider computational complexity of the node, lightweight authentication schemes e.g. ECDSA
Key exchange	Key encapsulation mechanisms which are PQC resistant, following the NIST standard (CRYSTALS-KYBER)	Key encapsulation mechanisms which may not be PQC resistant but are suitable for current security threats, e.g. RSA	Key encapsulation mechanisms which may not be PQC resistant but are suitable for current security threats and consider computational complexity of the node, lightweight key exchange schemes e.g. ECDSA



Hashing	At least 512 size hash, e.g. SHA-512	At least 256 size hash, e.g. SHA-256	Lightweight hashing algorithms, considering the computational complexity of the node, e.g. ASCON-Hash, QUARK, spongent, photon
---------	--------------------------------------	--------------------------------------	--

For example, if the underlying functionality is encryption: the functionality can be accessed by executing the encryption docker with appropriate command line arguments that specify a) security level b) throughput requirement c) power/energy requirement d) input files required (eg data/key file) e) any other information.

This will execute the encryption on the datafile specified using the key in the keyfile. The internal code of docker also tries to meet the various performance requirements requested through the command line arguments. For example if a medium security level requested, the docker automatically invoked AES-128 etc. An advantage of building each countermeasure as a docker is it allows for replacement of the security primitive, if in future its use is discontinued. In the unlikely event that AES 128 is discontinued in the future, one can perform a like for like replacement with any other encryption primitive that would replace it. This can simply be done by updating the encryption docker container.

Security primitives as service: To make the security modules available as a service, we intend to build a Security docker that will listen on a HTTP port for requests sent to each docker module, queues them and then parses them properly and forward the requests to the appropriate dockers. The Security docker then collects the results of each operation and forwards results to the appropriate senders. This will be made more efficient by using components of Kubernetes to seamlessly integrate it into the cluster infrastructure.

Current Status and relevant results: The focus of this WP is to develop the primitives that will be then used and instantiated in the Docker containers. We started with the AES algorithm, since it is the principal block cipher used for encryption (AES-128/192/256). If energy consumption of AES is well known for embedded and IoT devices, very little is known for servers. Addressing this limitation, we studied the effect of styles of implementation on energy consumption in a typical server environment. Energy was measured by reading system level registers present in the RAPL interface of an Intel Xeon i7 CPU E5-2660 v2 @ 2.20GHz with a total of 20 physical cores. We studied three implementation styles (see [Thakur-2024] for more details).

1. **Vperm** (Vector Permutation) implementation leverages vector operations, capitalizing on modern processors' Single Instruction, Multiple Data (SIMD) capabilities. By processing multiple data simultaneously, Vperm implementations significantly enhance the speed and efficiency of cryptographic operations.
2. **Table-Based Implementations** of cryptographic algorithms utilize precomputed tables to store intermediate values. These tables help speed-up computations, leading to improved performance. Commonly employed in symmetric ciphers, this approach enhances the performance of cryptographic operations by reducing the need for on-the-fly calculations.



3. **Bitsliced** implementations operate on multiple bits of data simultaneously, using bitwise operations to achieve high parallelism.

We present the findings in the following [Figure 4-1](#) and [Figure 4-2](#). In particular, implementation styles allowing for a high degree of parallelism were found to be useful for low energy deployments.

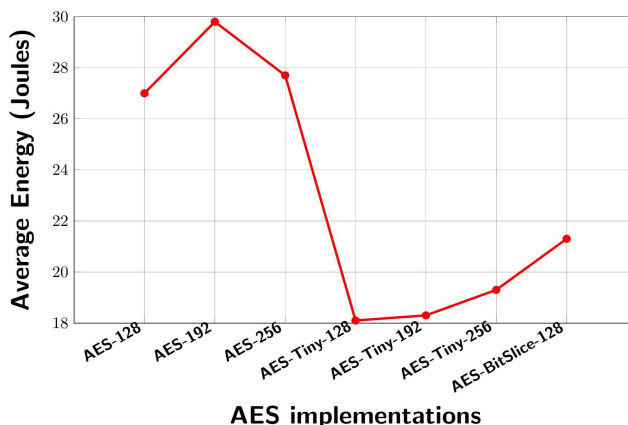


Figure 4-1: Energy consumption for AES family wrt different implementation styles [Thakur-2024].

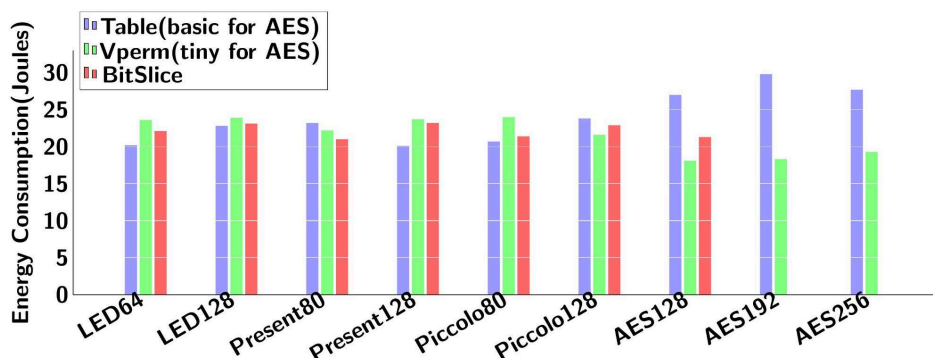


Figure 4-2: Comparison of energy consumption of AES with other lightweight block ciphers [Thakur-2024].

4.3 Gaia - X

Gaia-X aims to bring together a wide variety of services, each potentially built on different standards, architectures, or regulatory frameworks, and ensure that they can interoperate securely. Trust is the essential foundation for such interoperability in a federated environment, where participant identity must be verified, data integrity and security must be guaranteed, compliance and regulatory requirements must be met, and collaboration depends on verifiable commitments. To address these needs, Gaia-X has designed a Trust Framework [Gaia-x-22.04-Release-2024], which establishes an interoperable environment for data sharing and service federation by making any claims, from participant identity assertions to statements of regulatory conformance, both transparent and cryptographically verifiable.



In the following section, we briefly outline the design principles and core components of the Gaia-X Trust Framework, focusing on how it enables secure interoperability in federated environments through verifiable and cryptographically protected claims. We begin by detailing the architecture of the trust framework itself, including its use of Verifiable Credentials [W3-VC-DATA-MODEL-2025], SHACL-based validation policies [SHACL-TR-2025], and the layered structure of trust assurance levels. We include a description of the graduated trust tiers, ranging from basic legal identity verification to advanced compliance and sovereignty guarantees, along with the specific types of claims that can be validated at each level.

Building on this foundation, we present how the basic level of compliance (Tier 1), which establishes verified identity and minimal obligations, can be integrated into MYRTUS. Finally, we extend the discussion to explore the potential integration of Gaia-X trust mechanisms across other phases of the MYRTUS architecture, beyond Gaia-X Tier 1 level of trust.

4.3.1 Gaia-X Trust Framework: Technical Foundations

Gaia-X Trust framework is technically designed on the principle that all declared information must remain tamper-evident and traceable throughout its lifecycle. To achieve this objective, Gaia-X proposes, on a technical level, a verification strategy centered on the use of W3C Verifiable Credentials (VCs), each secured by a digital signature using Json Web Signature [JSON-Web-Signatures-2025]. Within a verifiable credential, it is possible to specify the claim itself, identify the entity making the claim and the entity who validated it. Any modification of the claim or its associated metadata would invalidate the signature, rendering such alterations immediately detectable. Consequently, the integrity and authenticity of a credential can be verified by any stakeholder with the appropriate verification keys. An integral part of Gaia-X's approach involves defining policy verification constraints, often written in the Shape Constraint Language (SHACL). Several open-source and commercial SHACL reasoning engines are available, such as the Eclipse RDF4J framework [RDF4J-2025], and Stardog [Stardog-2025]. The violation of these constraints can be dynamically checked at runtime with the use of a SHACL engine, a software component that interprets SHACL constraints to evaluate whether RDF-based data that can be invoked by the MIRTO cognitive engine, including Verifiable Credentials, meets specific structural and compliance requirements.

4.3.2 Gaia-X Trust Levels

Within the Gaia-X Trust Framework, trust is stratified into graduated assurance tiers, beginning with a basic identity-verified foundation and extending to stringent high-assurance levels. At the basic level, organizations must confirm their legal identity and meet a baseline set of compliance obligations to be certified as a rightful participant in the Gaia-X ecosystem.

Identity data, such as official registration details, can be represented through Verifiable Credentials, as previously addressed. SHACL shapes can then be applied to ensure the credential's structure aligns with Gaia-X policies (e.g., verifying the presence of mandatory fields). This process validates that an entity genuinely exists and is legally recognized before it can operate within the Gaia-X federation [Gaia-x-24.06-Release-2024] [Gaia-x-24.03-Release-2024]. This baseline establishes the minimum trust (and interoperability) required of all Gaia-X members.



Building on that foundation, the next tier corresponds to a “substantial” assurance level, which adds stricter requirements and verifications on top of the basic criteria. For example, at this level services must demonstrate adherence to privacy or data protection standards by presenting credentials that include relevant certifications (such as ISO or GDPR compliance). Gaia-X uses both automated checks (e.g., through a Digital Clearing House) and, where necessary, third-party audits to verify the credibility of these credentials. Validation rules, enforced through SHACL shapes, confirm that an organization’s credential data accurately reflects its compliance status.

Finally, the highest trust tier delivers “high” assurance, encompassing all prior requirements and introducing the most stringent constraints for maximum trust and sovereignty. A service at high assurance level inherits the identity verification and substantial-level controls, and must further ensure full EU jurisdictional oversight and robust auditability. In particular, providers at this top tier are generally required to be headquartered in the EU (to guarantee EU legal control over operations and data) and are subject to intensive external audits or certification processes for compliance. At this level, even data access by non-EU entities is effectively curtailed, reflecting a guarantee that sensitive data and processes remain under EU governance with a high degree of certainty. Each successive trust level in the Gaia-X framework thus builds upon the previous one, the basic level establishes a verified identity and compliance baseline, substantial level adds deeper verification and stricter policy conformance, and the high level imposes the toughest sovereignty and security requirements.

[Table 4-2](#) below summarizes the three primary assurance levels defined within the Gaia-X Trust Framework, each with its own set of obligations and validation processes.

Table 4-2: Gaia-X Trust Framework assurance level.

Trust Level	Description	Key Requirements
Basic (Tier 1)	Foundational identity assurance for all Gaia-X participants.	Identity verification of legal entity; Cryptographic signature of self-description as Verifiable Credential.
Substantial (Tier 2)	Enhanced assurance for sensitive or mission-critical services.	Stronger security and data protection controls; Evidence of certifications (such as compliance with GDPR, ISO 27001) or third-party attestations.
High (Tier 3)	Maximum assurance, focused on sovereignty and legal oversight.	All basic and substantial requirements; EU headquarters requirement; Restriction on non-EU access; Extensive external audits

4.3.3 Specific Application of the Gaia-X Trust Framework in MYRTUS

As detailed in [Section 3](#), MYRTUS intends to leverage cloud infrastructures that comply with the Gaia-X trust model framework. Gaia-X enables the establishment of a federated ecosystem of heterogeneous services, spanning Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS), provided by multiple entities, ensuring these services are verifiably secure and seamlessly integrable.



Within this context, MYRTUS proposes to integrate Gaia-X trust mechanisms at the Tier 1 level, corresponding to basic trust assurance. This tier requires verification of the legal identity of participating organizations and their conformity to a baseline set of compliance obligations. The decision to adopt Tier 1 is guided by the consideration that it currently constitutes the most stable, clearly defined, and technically supported element of the Gaia-X Trust Framework. Therefore, as Gaia-X continues to evolve, Tier 1 offers a reliable and well-established foundation for first integration.

Accordingly, the Gaia-X Trust Framework (Tier 1) can be applied in MYRTUS to verify the legitimacy of cloud providers and infrastructure services by examining their Verifiable Credentials, which prove their status as recognized participants in the Gaia-X ecosystem. These credentials confirm that the provider meets Gaia-X's baseline compliance requirements and has been issued a Verifiable Credential conforming to the basic-level trust tier.

As previously addressed, a Verifiable Credential is a digitally signed document that contains claims issued by one entity (the issuer) about another (the subject). These credentials are decentralized, meaning they can be verified without relying on the issuer; tamper-evident, due to cryptographic proofs; and interoperable, following W3C standards such as JSON-LD.

[Table 4-3](#) presents an example of Gaia-X Verifiable Credential, confirming that an entity is recognized as part of the Gaia-X trusted ecosystem after fulfilling the basic (Tier 1) identity verification level. [Table 4-3](#) presents a formatted breakdown of the key fields in a JSON-LD credential. Please note that in the fields, the "gx:" prefix indicates a Gaia-X ontological extension [Gaia-X-TrustFramework-2025] to the core Verifiable Credential data model. These fields, such as `gx:integrity`, `gx:version`, and `gx:type`, follow Gaia-X's trust framework and provide additional details, including document hashes (`gx:integrity`), the compliance framework version (`gx:version`), and the specific Gaia-X claim type (`gx:type`).

Table 4-3: example of Gaia-X Verifiable Credential.

#	Field	Value	Explanation
1	@context	"https://www.w3.org/2018/credentials/v1", "https://w3id.org/security/suites/jws-2020/v1", "https://registry.lab.gaia-x.eu/development/api/trusted-shape-registry/v1/shapes/jsonld/trustframework#"	Defines the JSON-LD context for interpreting the data.
2	type	VerifiableCredential	Indicates this document is a Verifiable Credential.
3	id	https://compliance.gxdch.gaiax.ovh/.../c1aa6a44-d915-...	Unique identifier for the credential instance.
4	issuer	did:web:compliance.gxdch.gaiax.ovh:v1	The entity (DID) that issued the credential.



5	issuanceDate	2025-04-17T14:17:05.555Z	The UTC timestamp when the credential was issued.
6	expirationDate	2025-07-16T14:17:05.555Z	The UTC timestamp when the credential expires.
7	credentialSubject	[...]	Array of entities and their compliance claims.
8	credentialSubject[0].type	gx:compliance	Indicates a compliance-related claim.
9	credentialSubject[0].id	https://www.delta-dao.com/.../gx_participant.json	Points to a participant description document.
10	credentialSubject[0].gx:integrity	sha256-8325ffb0301e4c...	Hash of the referenced JSON for integrity.
11	credentialSubject[0].gx:integrityNormalization	RFC8785:JCS	JSON canonicalization method used for hashing.
12	credentialSubject[0].gx:version	22.10	Compliance framework version.
13	credentialSubject[0].gx:type	gx:LegalParticipant	Denotes the subject is a legal participant.
14	credentialSubject[1].type	gx:compliance	Another compliance-related claim.
15	credentialSubject[1].id	https://www.delta-dao.com/.../gx_registrationnumber.json	Points to the legal registration number.
16	credentialSubject[1].gx:integrity	sha256-e9ac030e0070b22...	Integrity hash for registration document.
17	credentialSubject[1].gx:integrityNormalization	RFC8785:JCS	JSON canonicalization method.
18	credentialSubject[1].gx:version	22.10	Compliance framework version.
19	credentialSubject[1].gx:type	gx:legalRegistrationNumber	Specifies the type of registration claim.



20	credentialSubject[2].type	gx:compliance	Another compliance-related claim.
21	credentialSubject[2].id	https://www.delta-dao.com/.../gx_tandc.json	Points to the Terms and Conditions document.
22	credentialSubject[2].gx:integrity	sha256-0e61906a39902224...	Integrity hash for Terms & Conditions document.
23	credentialSubject[2].gx:integrityNormalization	RFC8785:JCS	Normalization method.
24	credentialSubject[2].gx:version	22.10	Version identifier.
25	credentialSubject[2].gx:type	gx:GaiaXTermsAndConditions	Claim type: Terms and Conditions.
26	proof	[...]	Holds cryptographic signature and metadata.
27	proof.type	JsonWebSignature2020	Specifies the signature format.
28	proof.created	2025-04-17T14:17:05.910Z	When the proof was created.
29	proof.proofPurpose	assertionMethod	Indicates the purpose of the proof.
30	proof.verificationMethod	did:web:...#X509-JWK2020	Where to find the public key to verify signature.
31	proof.jws	eyJhbGciOiJIUzI1NiIsImI2NCI6...	The actual digital signature (truncated).

To verify the presence of Gaia-X Verifiable Credentials, the constraints presented at [Table 4-3](#) have been designed to be used in MYRTUS. These constraints can be executed as SHACL shapes against the data graphs of Verifiable Credentials to validate that the required information is present for the Tier 1 level of trust.

Within the SHACL shapes designed to validate Gaia-X Verifiable Credentials (Tier 1), the following namespace prefixes are used in the table. The `sh:` prefix links to the SHACL namespace [SHACL-NS-2025], while `xsd:` aligns with XML Schema datatypes [XMLSchema-2025]. The `cred:` prefix captures the W3C Verifiable Credentials vocabulary [VerifiableCredentials-2018], and `gx:` references the Gaia-X trust framework [Gaia-X-TrustFramework-2025]. Additionally, the `sec:` prefix points to security-related terms in the Security Vocabulary [W3C-Security-2025], and `ex:` serves as an example for shapes instantiation [ExampleShapes-2025].

Table 4-4: MYRTUS constraints to verify presence of Gaia-X Verifiable Credentials.



Shape	Constraint	SHACL
GaiaXCredentialShape	Must be typed as VerifiableCredential	<pre> sh:property [sh:path rdf:type ; sh:hasValue cred:VerifiableCredential ;] ; </pre>
GaiaXCredentialShape	Must have issuer as a string	<pre> sh:property [sh:path cred:issuer ; sh:datatype xsd:string ; sh:minCount 1 ;] ; </pre>
GaiaXCredentialShape	Must have an issuanceDate	<pre> sh:property [sh:path cred:issuanceDate ; sh:datatype xsd:dateTime ; sh:minCount 1 ;] ; </pre>
GaiaXCredentialShape	Must have an expirationDate	<pre> sh:property [sh:path cred:expirationDate ; sh:datatype xsd:dateTime ; sh:minCount 1 ;] ; </pre>
GaiaXCredentialShape	Must have at least one credentialSubject of type GaiaXCredentialSubjectShape	<pre> sh:property [sh:path cred:credentialSubject ; sh:minCount 1 ; sh:node ex:GaiaXCredentialSubjectShape ; sh:nodeKind sh:BlankNodeOrIRI ;] ; </pre>
GaiaXCredentialShape	Must have a proof object conforming to GaiaXProofShape	<pre> sh:property [sh:path sec:proof ; sh:minCount 1 ; sh:node ex:GaiaXProofShape ;] ; </pre>
GaiaXCredentialSubjectShape	rdf:type must be gx:compliance	<pre> sh:property [sh:path rdf:type ; sh:hasValue gx:compliance ; sh:minCount 1 ;] ; </pre>
GaiaXCredentialSubjectShape	gx:type must be one of defined Gaia-X types	<pre> sh:property [sh:path gx:type ; sh:in (gx:LegalParticipant gx:legalRegistrationNumber gx:GaiaXTermsAndConditions) ; sh:minCount 1 ;] ; </pre>
GaiaXCredentialSubjectShape	gx:integrity must be a string	<pre> sh:property [sh:path gx:integrity ; </pre>



		<pre> sh:datatype xsd:string ; sh:minCount 1 ;] ; </pre>
GaiaXCredentialSubjectShape	gx:integrityNormalization must be a string	<pre> sh:property [sh:path gx:integrityNormalization ; sh:datatype xsd:string ; sh:minCount 1 ;] ; </pre>
GaiaXCredentialSubjectShape	gx:version must be a string	<pre> sh:property [sh:path gx:version ; sh:datatype xsd:string ; sh:minCount 1 ;] ; </pre>
GaiaXProofShape	rdf:type must be JsonWebSignature2020	<pre> sh:property [sh:path rdf:type ; sh:hasValue <https://w3id.org/security#JsonWebSignature2020> ; sh:minCount 1 ;] ; </pre>
GaiaXProofShape	created must be a dateTime	<pre> sh:property [sh:path sec:created ; sh:datatype xsd:dateTime ; sh:minCount 1 ;] ; </pre>
GaiaXProofShape	proofPurpose must be 'assertionMethod'	<pre> sh:property [sh:path sec:proofPurpose ; sh:in ("assertionMethod") ; sh:minCount 1 ;] ; </pre>
GaiaXProofShape	verificationMethod must be a string	<pre> sh:property [sh:path sec:verificationMethod ; sh:datatype xsd:string ; sh:minCount 1 ;] ; </pre>
GaiaXProofShape	jws must be a string	<pre> sh:property [sh:path sec:jws ; sh:datatype xsd:string ; sh:minCount 1 ;] ; </pre>

As shown in [Table 4-4](#), the constraints of the credential validation model designed by MYRTUS are defined through three distinct SHACL shapes, each targeting a specific component within the RDF data graph of a Verifiable Credential. These shapes are intended to ensure that all structural, datatype, and semantic constraints imposed by the Gaia-X Trust Framework for Tier 1 are satisfied. Hereunder, the constraints embedded in the shapes are described.

The *GaiaXCredentialShape* targets the credential node, identified by the class `cred:VerifiableCredential`. It validates the presence and structure of core properties such as `cred:issuer`, `cred:issuanceDate`, and



`cred:expirationDate`, and ensures that both `cred:credentialSubject` and `sec:proof` are present and conform to their respective nested validation shapes.

The *GaiaXCredentialSubjectShape* validates each credential subject node (i.e., the entities described within the `cred:credentialSubject` array). It enforces that the subject is typed as `gx:compliance` and that the `gx:type` property holds a value from the allowed Gaia-X compliance classes (`gx:LegalParticipant`, `gx:legalRegistrationNumber`, `gx:GaiaXTermsAndConditions`). Additionally, the shape checks for the presence and datatype correctness of `gx:integrity`, `gx:integrityNormalization`, and `gx:version`.

The *GaiaXProofShape* applies to the proof node embedded in the credential. It enforces conformance with the `JsonWebSignature2020` signature suite, requiring properties such as `sec:created` (timestamp), `sec:proofPurpose` (which must be "assertionMethod"), `sec:verificationMethod`, and `sec:jws` (the compact digital signature string).

In the MYRTUS architecture, the validation of Gaia-X Verifiable Credentials can take place within a dedicated microservice (running a SHACL engine), referred here as a "Compliance Check Service". This microservice will be deployed in a containerized fashion, ensuring it can be scaled or updated independently. The MIRTO agent can invoke this service (e.g., via a REST API) to confirm compliance of the verifiable credentials provided by the entities in the MYRTUS ecosystem before granting resource access or updating the knowledge base.

4.3.4 Further Evaluation of the Gaia-X Compliance Trust Mechanism for Extended Functionalities in MYRTUS

In order to assess the full potential of Gaia-X's Trust Framework for the MYRTUS Reference Infrastructure, we have evaluated the feasibility of applying the Gaia-X Trust Framework beyond the basic credential-verification stage to additional phases of the MYRTUS architectural process. This exploration goes beyond verifying whether an entity holds a Gaia-X-issued Verifiable Credential and instead considers how an expanded trust architecture, potentially incorporating the Gaia-X reference model and enhanced ontologies, could unify broader compliance requirements across the MYRTUS ecosystem. This work is carried out with consideration of the ongoing developments within the Gaia-X Trust Framework, acknowledging that certain components and specifications are still evolving. The current effort is thus designed with extensibility in mind, ensuring that MYRTUS remains aligned with the broader Gaia-X trajectory. Tiers 2 and 3 (levels of Trust) will enable the verification of claims (VCs) regarding software and infrastructure components. The verification of these components will take place at different times during the lifecycle of the MYRTUS application, the MIRTO cognitive engine, and even the framework itself.

Within the scope of the MYRTUS project, the Gaia-X Trust Framework can be operationalized to reinforce continuous compliance and trust across a heterogeneous computing continuum, spanning edge, fog, and cloud resources. MYRTUS's MIRTO cognitive engine dynamically orchestrates workload distribution based on performance metrics, security trust levels, and energy efficiency. By embedding Gaia-X trust and compliance controls directly into the architectural layers, MYRTUS gains the ability to reliably attest to, and enforce, compliance checks for diverse infrastructure components, acknowledging the varying provenance of data



and services. In practice, this would entail extending the existing “Compliance Check Service”, the SHACL-based verification microservice described earlier, to incorporate additional shapes beyond the baseline Gaia-X requirements for verification of basic trust level (tier 1).

In particular, each MYRTUS component, whether an edge-based microservice handling real-time sensor streams or a cloud-resident analytics engine, can be equipped with Verifiable Credentials that attest to relevant properties like GDPR compliance or ISO 27001 certification. Under the Gaia-X reference architecture, such certifications would typically classify a participant at Substantial (Tier 2) or High (Tier 3) trust levels. Whenever the MIRTO engine initiates a task deployment or data transfer, it can validate these credentials against predefined policies, confirming that the target node meets the required compliance posture. These credential checks occur automatically through SHACL-codified policy rules, enabling the orchestration logic to make informed, context-specific decisions without manual oversight.

As an illustrative example, consider a scenario in which sensitive data can only be processed by nodes located in a particular legal jurisdiction or by services certified for secure data handling. SHACL-based policies can enforce these constraints, restricting eligible nodes to those meeting jurisdictional and security requirements, thereby minimizing compliance risk by preventing any unauthorized or non-compliant resource from participating in the workflow.

To further enhance accountability, MYRTUS can incorporate Gaia-X-style audit trails, capturing each credential verification, policy evaluation, and data exchange event. By implementing tamper-evident techniques, these logs remain both immutable and independently verifiable, ensuring that automated systems and human auditors alike can retrospectively confirm whether each operational decision met the stipulated policies and regulatory obligations. Consequently, we consider that a Gaia-X-driven trust mechanism, extended beyond basic credential checks, may provide a comprehensive foundation for safeguarding compliance and integrity across all layers of the MYRTUS architecture in the future.

4.4 Distributed Knowledge Base

4.4.1 Description

To create a continuum of information, all the layers share a unified ontological knowledge base, where the data is distributed across different layers used for both data analysis and informed decision making. The DKB is a system that stores and shares knowledge across multiple distributed databases within the MYRTUS continuum ensuring data consistency and availability across the system. The DKB acts as a central repository for all the data within MYRTUS.

All the monitored metrics (such as cpu, memory, energy utilisation, etc.) collected by components from the infrastructure and the corresponding resource status information (such as available compute capacity, memory, etc.) at each layer will be available via the resource registry and stored in the DKB. Different managers developed in MIRTO Cognitive Engine (WP5) can access information about resources stored in the DKB to make deployment and workload redistribution decisions.



The DKB provides a standard interface as described in [Section 4.4.2](#) for storing and retrieving information. This helps all the components to access and store the information in a unified manner thereby enabling data sharing between all components present in different layers.

In order to support various data formats in the DKB (such as JSON/YAML, PyTorch file, key-value, binary objects, time series data, etc.), different open source distributed database technologies are considered, as described in [Table 4-5](#),

Table 4-5: open source distributed databases.

Database	Description	Best Use case
ETCD [Etc-d-2025]	Default key-value store for Kubernetes cluster state.	Kubernetes cluster state and configurations.
Redis [Redis-2025]	In-memory key-value store with clustering support.	Caching and transient data in Kubernetes.
Apache Cassandra [Cassandra-2025]	Highly scalable distributed NoSQL key-value store.	Large-scale metadata and time-series data.
MongoDB [MongoDB-2025]	NoSQL document-oriented database designed for flexibility and scalability.	Storing semi-structured data like logs or configuration documents for Kubernetes applications.
Minio - Object Storage [Minio-2025]	Open-source, high-performance, distributed object storage system	Ideal for storing large-scale unstructured data such as logs, backups, media files, JSON/YAML documents, and PyTorch model files.
Cockroachdb [Cockroachdb-2025]	Cloud-native distributed SQL database designed for scalability and resilience.	Suitable for applications needing SQL support with high availability and horizontal scaling.
InfluxDB/TimeScale : time series DB [InfluxDB-2025] [Timescale-2025]	A time series database (TSDB) is a database optimized for time-stamped or time series data	Suitable to analysis purpose for time stamped data

Storing all kinds of data is challenging and not possible using a single technology, so we have decided to choose multiple database options to develop the initial version of the DKB for MYRTUS. We consider

- **Minio** for storing binary objects and files,
- **ETCD** for key-value data, and
- **MongoDB** for unstructured data such as Json.

This multi-database approach ensures that the DKB can meet the diverse storage and access requirements associated with different data formats.



At this stage, HIRO works on the implementation of the DKB, while ABI assesses the communication with MongoDB (implemented in a node in the ABI SenseIQ gateway) and with an external ETCD (see [Section 5.1.2.2](#)).

4.4.2 Interfaces

MYRTUS is designed as a modular service-based architecture; different components expose standardized OpenAPI REST [OpenAPI-2025] interfaces. These standardized interfaces help in development of components independently ensuring communication between different components seamlessly. In the following, we will provide the two interfaces that are being developed within WP3.

4.4.2.1 DKB Interface

DKB provides a standard unified interface for storing and retrieving information. APIs are defined based on standard REST OpenAPI specification. The API provides the general interaction with regard to the namespaces and objects. Namespace is the logical container that helps organize and manage data storage efficiently. To ensure seamless interoperability, DKB allows different components to send and retrieve data in different formats (binary objects, JSON, and YAML) without compatibility issues.

The initial version of OpenAPI specification for DKB interface is shown in [Figure 4-3](#).

These are the operations provided in initial version of the DKB:

- Creation of the namespace: creates a logical entity
- Deletion of the name space: deletes the logical entity
- Uploading the object to the namespace: uploads objects in the logical entity
- Getting the object from the namespace: retrieves objects in the logical entity
- Deletion of the object from the namespace: deletes objects in the logical entity

Myrtus Knowledge Base API			^
PUT	/namespace/{namespace_name}	Create a namespace	✓
DELETE	/namespace/{namespace_name}	Delete a namespace	✓
PUT	/namespace/{namespace_name}/objects/{object_key}	Upload an object	✓
GET	/namespace/{namespace_name}/objects/{object_key}	Retrieve an object	✓
DELETE	/namespace/{namespace_name}/objects/{object_key}	Delete an object	✓

Figure 4-3: D3.1 DKB Interface specification.

4.4.2.2 Resource Registry Interface

The resource registry interface is dedicated towards providing the actual state of the MYRTUS system. It distributes information on the current status of the MYRTUS system, providing a



comprehensive view of resource availability and usage. This information is mainly used by the MIRTO Agents to make informed decisions, like workload allocation. The initial version of OpenAPI specification for resource registry interface is shown in [Figure 4-4](#):
Following are the operations provided in initial version of resource registry:

- Resources
 - Get all information of the current status in the realization of a MYRTUS Reference Infrastructure.
- Network
 - List all networks in the realization of a MYRTUS Reference Infrastructure.
 - Get the nominal information of a specific network.
- Compute
 - List all the compute nodes in the realization of a MYRTUS Reference Infrastructure.
 - Get the nominal information of a specific compute node.

MYRTUS resource registry API OAS 3.0

resources		^
GET	/resources	▼
network		^
GET	/network	▼
GET	/network/{name}	▼
compute		^
GET	/compute	▼
GET	/compute/{name}	▼

Figure 4-4: D3.1 Resource registry Interface specification.

5 Implementation and Integration

This section details the progress on specific components, starting from what was planned in D1.1 “UC Needs, requirements and KPIs v1” for Pillar 1 results, to introducing all the already available partial integrations of continuum infrastructures made available for preliminary assessment.

5.1 Implementation

5.1.1 Edge Components



5.1.1.1 HMPSoCs FPGA-based accelerators

[Table 5.1](#) introduces the FPGA-based HMPSoCs accelerators that leverage open-source architectural templates and employ Coarse Grained Reconfigurable (CGR) techniques to enhance edge computing capabilities.

Table 5-1: Synopsis of the HMPSoCs FPGA-based accelerators component.

HMPSoCs accelerators			
ID: R1	Licence: 3-clause BSD	Owner: UNICA UNISS	Contacts: francesca.palumbo@unica.it francesco.ratto@unica.it claudio.rubattu@uniss.it
Short Description: HMPSoCs FPGA-based accelerators leveraging on open-source architectural templates, and exploiting coarse-grained reconfigurable (CGR) approaches for edge computation.			
Require	- RTL description of the CGR accelerator and the RTL co-processor template - Programming tables - APIs <i>All these inputs are derivable with the MDC tool that is part of MYRTUS DPE.</i>		
Provide	Acceleration infrastructure at the edge.		
Input	Binary files for the CGR coprocessor (configuration bitstream), software application instrumented with the APIs to utilize the co-processing unit.		
Output	Heterogeneous and hardware-accelerated execution of the application.		
MYRTUS layer	Edge		
TRL@M0	3		
TRL@M36	5		

[Figure 5-1-1](#), [Figure 5-1-2](#) and [Figure 5-1-3](#) represent on the left hand side the different input dataflow specification and, on the right hand side, the resulting acceleration datapath along with execution examples. Multithreading allows concurrent execution, while multitasking interleaved ones. Thus, functionally, the dataflow-driven CGR accelerators developed by UNISS and UNICA enable:

- Multitasking execution at the edge, supporting both functional multitasking ([Figure 5-1-1](#)) —where the accelerator is dynamically reconfigured to perform different tasks— and trade-off-oriented multitasking ([Figure 5-1-2](#)), allowing adaptation to different task implementations based on performance or resource constraints.
- Multithreading execution ([Figure 5-1-3](#)), intended as the ability to process concurrently multiple instances, *threads*, executing the same functionality concurrently, e.g., executing an encryption algorithm on a set of independent blocks. In a CGR system this can result in an effective execution that better exploits the customized deep pipeline.

This dataflow approach extends also to AI acceleration at the edge, utilizing a streaming-based methodology to optimize execution efficiency and enable adaptivity.

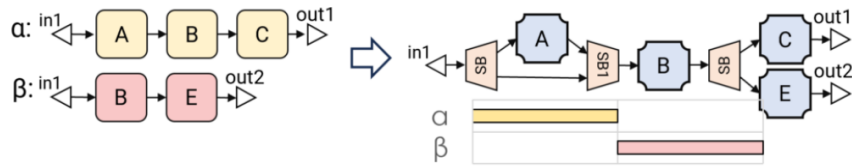


Figure 5-1-1: Functional reconfigurability. The coprocessor can be reconfigured to execute the desired functionality.

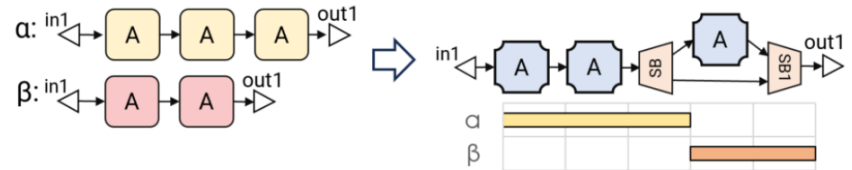


Figure 5-1-2: Non-functional reconfigurability. The coprocessor can be reconfigured to execute the desired implementation of the functionality.

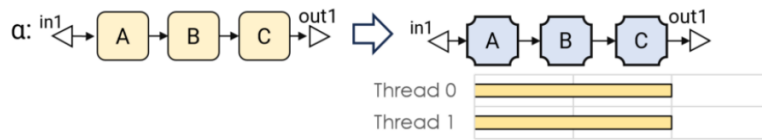


Figure 5-1-3: Multithreading. The coprocessor supports the concurrent execution of multiple threads

R1, as demonstrated in [Table 5-2](#), is compliant with EU-CEI building blocks.

Table 5-2: EU-CEI Mapping of the HMPSoCs FPGA-based accelerators Component.

EU-CEI Block	UNICA-UNICA EDGE HMPSoC Accelerators
Security and Privacy Trust and Reputation	Already available support for AES128 and AES256 encryption/decryption. During the second half of the project we have planned to support all three security levels (see Table 4-1).
Data management	Local storage in main memory and processing by accelerators is enabled and the DKB support with API-based management is ongoing. In particular, we already have preliminary APIs capable of reaching the DKB via the ABI's gateway (described in Section 5.1.2.2 below) with the purpose of writing and reading data concerning the execution profile and monitored data.
Resource management Orchestration	Local resource management and orchestration will leverage a custom Linux distribution built with Yocto that enables K3s service. K3s is the lightweight version of K8s, more suitable for edge computing nodes, compatible with K8s. Ligo support has been integrated to handle multi-cluster management (see Section 5.2.1 below).
Network	Different communication protocols can be supported through customization of adopted Operating System (OS) distribution. For example standard protocols like HTTP (used for edge-fog integration presented in Section 5.2.2 below) are already supported. In the past, TCP/UDP and MQTT were also adopted.
Monitoring and Observability	Power consumption and performance monitoring is available. Retrieved data will be made available in the DKB.
Artificial Intelligence	The toolchain developed in Pillar 3 for Convolutional Neuroal



	Networks (CNNs) can be used to enable AI acceleration using a streaming-based methodology.
--	--

During the first year of the project the following advancements in the component have been made:

- HMPSoC Accelerators support **approximate computing** in the scope of ML acceleration [Manca-2024]. This contributes to reduced power consumption (with respect to full precision computation) and enables adaptivity at hardware levels by allowing a runtime reconfiguration of the adopted precision. This type of accelerators can be derived with an automated toolchain developed in Pillar 3. Baseline examples are available on the same MDC repository [MDC-Tool-2025] .
- The **multithread** accelerators architecture have been defined and implemented. Some example accelerators for AES, FFT and HEVC filtering have been derived and will be released (currently under revision for publication).
- The **custom Linux distribution** build for the target devices have been enhanced with K3s and Liqo support to allow integration in the reference architecture.
- Power consumption and performance **monitoring** have been enabled with a monitoring framework developed with UPM.
- The **DKB support** with API-based management is ongoing.

The advancements have been assessed as follows:

- The approximate computing support for CNNs has been assessed on an MNIST accelerator. The accelerator is demonstrated to reach state-of-art performance while guaranteeing up to 7% energy saving in its less-accurate configuration [Manca-2024].
- The multi-threading support has been assessed on various kernels (FFT on various time windows, AES encryption, HEVC filter). For these kernels acceleration with respect to multithread software execution on the HeMPSoC has been shown, with speed ups varying between 7x and 41x depending on the application. All these accelerators have been derived through an automated design methodology applicable to any other dataflow-based kernel to be accelerated at the edge.
- The custom Linux distribution and the integration with the fog layer through ABI gateway has been assessed with the exchange of AES-encrypted data.

Overall these contribution concur to achieve MYRTUS objectives **T1.OO.a**, as the accelerators can be integrated and monitored within a layered computing continuum infrastructure, and **T1.OO.b**, as the accelerators provide increased energy efficiency while preserving flexibility of functionality and adaptivity of execution metrics.

The assessment of the described developments, along with the edge-to-edge integration ([Section 5.2.1](#)) and the fog-to-edge integration ([Section 5.2.2](#)), demonstrates the component's contribution toward achieving the project's KPIs. Specifically, **KPI1.1** is supported by the custom OS and the integration of Kubernetes and Liqo on the edge HeMPSoCs, while **KPI1.2** and **KPI1.3** are addressed through the demonstrated performance and efficiency gains achieved via hardware multithreading and approximate computing.

5.1.1.2 Multi-Grain CGRA + RISC-V Integration



[Table 5-3](#) shows the general overview of the RISC-V processors with CGRA-based extensions to be used as one of the computing flavors at the edge in MYRTUS.

Table 5-3: Synopsis of the Adaptive RISC-V processor with custom CGRA-based extensions component.

Adaptive RISC-V processor with custom CGRA-based extensions [UPM]			
ID: R2	Licence: Solderpad Hardware Licence v2.1	Owner: UPM	Contacts: joseandres.otero@upm.es alfonso.rodriquezm@upm.es
Short Description: Multi-grain Coarse-Grain Reconfigurable Array (CGRA) accelerator coupled with an open-source RISC-V processor through custom ISA extensions (tight) and/or MMIO interfaces (loose) to support 1) automatic SW-to-HW code offloading using HW composition, 2) edge AI capabilities in the RISC-V ecosystem.			
Require	Application code with hardware acceleration requirements at the edge.		
Provide	Acceleration infrastructure at the edge.		
Input	Binary files for processor (application executable), CGRA accelerator (CGRA configuration bitstream), and the static configuration bitstream for the FPGA.		
Output	Heterogeneous and hardware-accelerated execution of the application.		
MYRTUS layer	Edge		
TRL@M0	3-4		
TRL@M36	5		

The main purpose of the RISC-V + CGRA edge nodes is to enable heterogeneous computing and automatic acceleration offloading from software to hardware. This procedure involves infrastructure components, design-time tools and run-time mechanisms. This section will address the first one, while the other two will be covered in D7.1 and D5.1, respectively.

The CGRA accelerator used in this component is a spatially distributed CGRA that uses elastic logic (i.e., valid/ready signal pairs for each data signal) to avoid time-multiplexing of instructions (i.e., once configured, the accelerator can be used without requiring additional reconfigurations during the computation of a given kernel, except to completely change the implemented functionality or to adapt to different non-functional requirements) and to enable dynamic scheduling in the dataflow (i.e., the hardware itself ensures that the alternative datapaths in the offloaded data-flow graph (DFG) are synchronized without compiler or programmer intervention).

The CGRA is laid out as a 2D structure of Processing Elements (PEs) with neighbor to neighbor connectivity to keep the delay in the critical path (and thus, the maximum operating frequency of the accelerator) under reasonable values. The structure of each PE, which includes an ALU (including addition, subtraction, multiplication, bit shifts, and, or and xor operations) and the logic required to enable conditional paths and branches (if-else operations), is shown in [Figure 5-2](#).

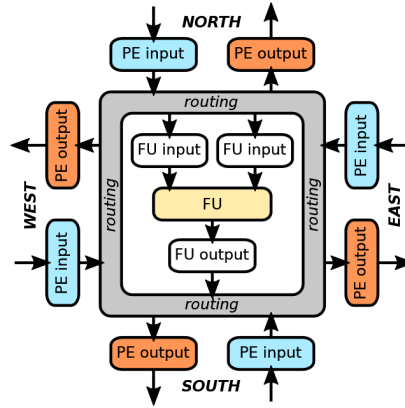


Figure 5-2: CGRA Processing Element structure. PE: Processing Element. FU: Functional Unit.

Depending on the complexity of the application section, or kernel, to be offloaded onto the CGRA accelerator, different spatial mappings can be supported:

1. If the kernel is small but can be parallelizable, loop unrolling techniques can be applied to replicate the datapath or DFG in the CGRA and support data-level parallelism.
2. If the kernel is too large to fit the available number of PEs, its DFG can be divided into several sub-DFGs, and then the application computes each of them for all input data before moving onto the next one (thus minimizing the reconfiguration overhead).
3. If the kernel does not support data-level parallelism, or the amount of consumed resources is close to the maximum number of PEs, a single DFG can be mapped.

All these scenarios can be seen in [Figure 5-3](#), where the manual mappings of a ReLU (mapping 1), an FFT (mapping 3) and a matrix multiplication (mapping 2) kernels is shown.

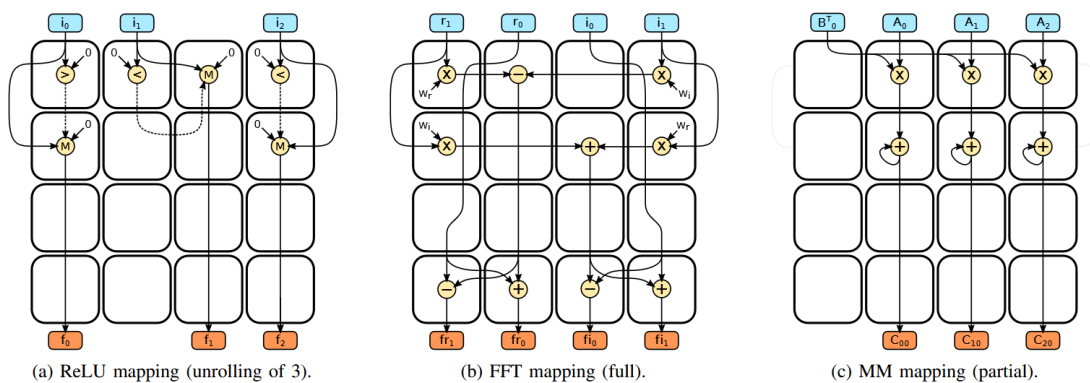


Figure 5-3: Spatial mappings of different kernels on a 4x4 CGRA depending on their DFG complexity.

During the period being reported in this deliverable, there have been several advances with regard to the adaptive RISC-V processors with custom CGRA-based extensions. The first and most important one has been the **integration of the baseline CGRA accelerator**, which was already available for microcontroller-grade RISC-V systems before the beginning of the project,



within an **System on Chip (SoC)** that includes the **CORE-V CVA6 application-grade processor** [CVA6-2025] . [Figure 5-4](#) shows the top-level view of the block diagram of the resulting SoC.

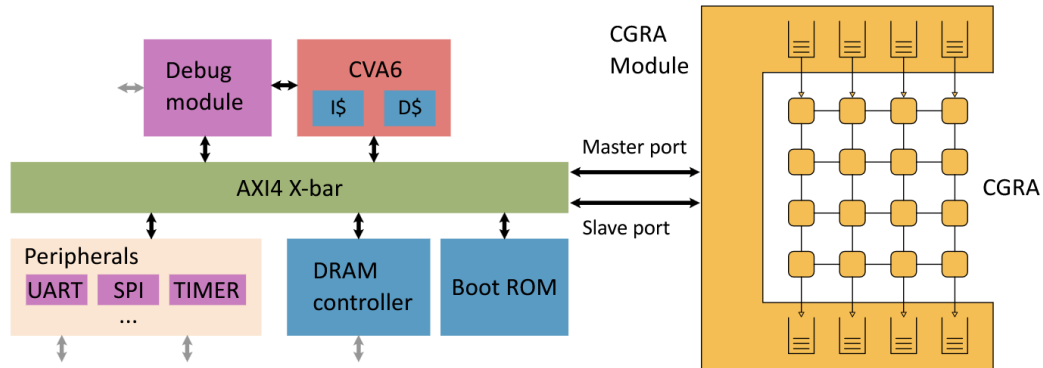


Figure 5-4: Integrated CORE-V CVA6 RISC-V processor + MMIO CGRA accelerator.

For this integration, several architectural changes have been made in the wrapper logic that connects the core CGRA structure with the buses of the SoC. The initial version of the CGRA was integrated with X-HEEP [Vazquez-2024], a platform also built around a CORE-V processor (CVE3240P in this case) but with an interleaved multi-port on-chip memory system and Open Bus Interface (OBI) bus interconnections. As a result, the CGRA wrapper included several parallel DMA nodes to obtain the maximum computing performance. The target CVA6 SoC, on the other hand, has an external RAM memory (as the data storage requirements in application-grade processors are significantly larger) with a single-port memory controller and AXI-based interconnections. This has required the **modification of the CGRA accelerator wrapper** shown in [Figure 5-5](#). As it can be seen, the modified wrapper logic uses **single DMA-capable channels to read from and write to the AXI infrastructure**, while at the same time enabling **multiplexed access to the parallel input/output ports of the core CGRA module**. In addition, **GNU/Linux support for the MMIO CGRA accelerator** has also been developed to enable the future integration of this node flavor in the MYRTUS Reference Infrastructure. This work has been the result of a Master Thesis associated with the project [Granja-2024], and has been made open source in UPM-CEI's GitHub organization [StrelaCVA6-2025].

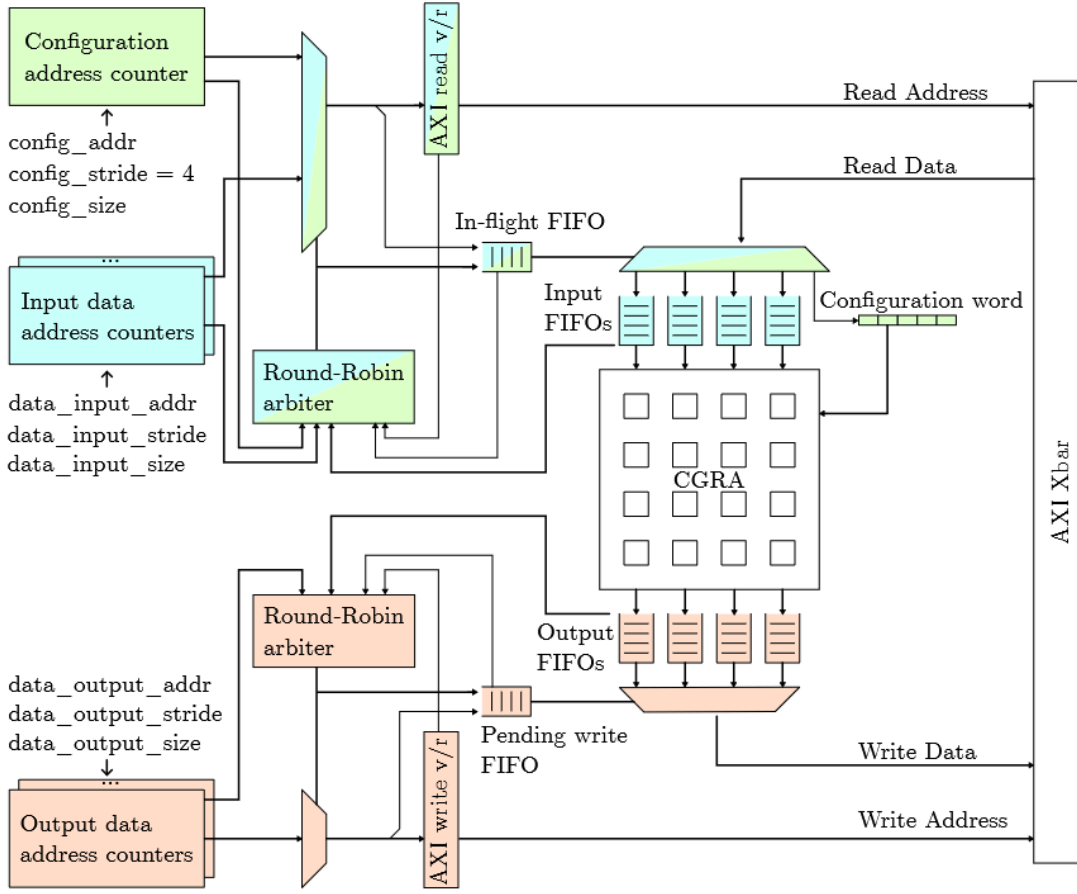


Figure 5-5: Modified wrapper logic for the AXI-based MMIO CGRA accelerator.

Apart from the main computing elements of the RISC-V + CGRA component, UPM has also worked together with UNICA and UNISS on **extending the baseline monitoring infrastructure**, brought into MYRTUS as previous background [Encinas-2024], to support **run-time power and computing performance trace acquisition for multiple hardware-accelerated edge, fog and cloud node** configurations. UPM has also worked on creating power consumption and performance predictive models for the edge nodes, at run-time, based on the data obtained by this monitoring infrastructure [Encinas-2025]. Detailed results for these models are provided in D5.1.

As the integration of the CGRA accelerator in the CVA6-based SoC and the extension to the baseline monitoring infrastructure happened in parallel, an already existing and mature platform available at UPM-CEI was used to also advance in connecting the adaptive FPGA-based edge nodes with the rest of the MYRTUS infrastructure. This strategy has allowed the research team to obtain some preliminary results, also in this regard. The target platform, ARTICo³ [ARTICo³-2025], also provides adaptive hardware acceleration support for applications with task- and data-level parallelism, and relies on a GNU/Linux operating system [ARTICo³-2025]. This alternative platform has been used to **prototype and deploy the initial virtualization support for heterogeneous reconfigurable systems with Kubernetes (K3s) and**



Liqo, targeting the integration of this edge node flavor in the reference MYRTUS infrastructure. [Figure 5-6](#) shows a high-level overview of a proof-of-concept setup that connects **several FPGA-based devices with different amounts of computing resources to mimic the edge (Pynq-Z1 boards), fog (ZCU102 boards) and cloud (Alveo U250 boards)**. This work has been carried out in another Master Thesis, and has led to the publication of a conference paper [Diez-de-Ulzurrun-2024].

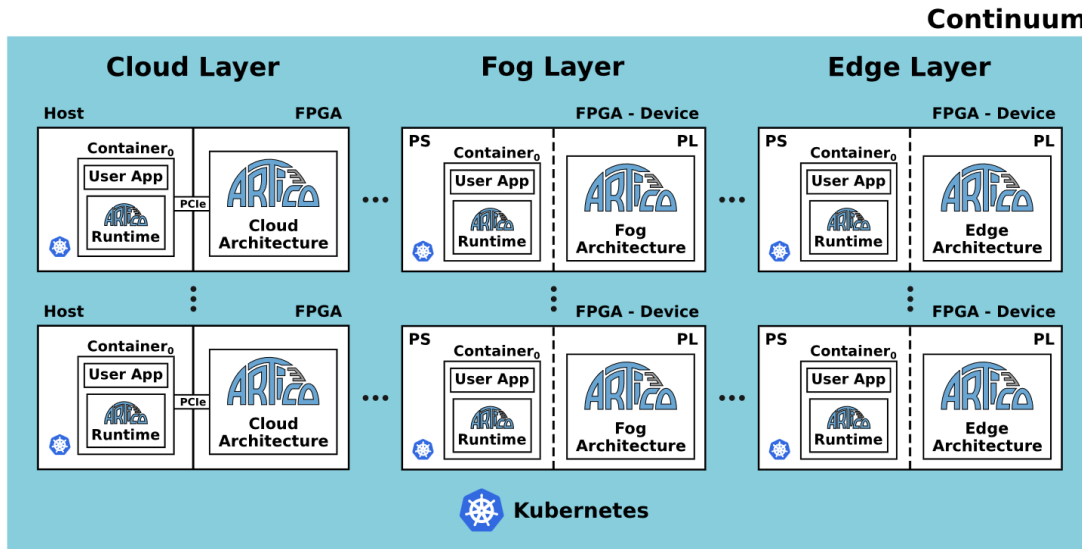


Figure 5-6: K3s-enabled continuum of heterogeneous reconfigurable accelerators.

All these technologies have been experimentally validated and assessed as follows:

- The CGRA + RISC-V SoC has been tested using manually-mapped kernels from the PolyBench benchmarking suite and RTL simulations. FPGA synthesis results have been also used to obtain resource utilization metrics.
- The virtualization support with K3s has been evaluated in actual hardware setups using several FPGA nodes (Pynq-Z1, ZCU102, Alveo U250) and different kernels from the MachSuite benchmarking suite. This setup has been also used to demonstrate the Liqo capabilities to connect this infrastructure with the rest of the MYRTUS infrastructure. This latter setup has been also used to validate the portability of the monitoring mechanisms and tools.

The relationship between the work reported here and the MYRTUS Operational Objectives and KPIs can be summarized as follows:

- The generalizable **monitoring infrastructure** is associated with **T1.00.a**, and will lay the foundation for the assessment of **KPI1.2** and **KPI1.3**.
- The integration of the adaptive **CGRA accelerator with the CVA6** application-grade processor, together with the GNU/Linux OS support contributes to **T1.00.b** and **KPI1.1**, and will be used to evaluate **KPI1.2** and **KPI1.3**.
- The integration of **lightweight Kubernetes (K3s) and Liqo for FPGA-based edge nodes** contributes to **T1.00.a** and **KPI1.1**.

Finally, [Table 5-4](#) shows how R2 is mapped within each of the EU-CEI building blocks.



Table 5-4 - EU-CEI Mapping of the Adaptive RISC-V processor with custom CGRA-based extensions component.

EU-CEI Block	UPM RISC-V processor with custom CGRA-based extensions
<i>Security and Privacy Trust and Reputation</i>	Additional Memory-mapped Input/Output (MMIO) hardware accelerators for AES (128, 192, 256) and SHA-2 (256, 384, 512) are already available to be integrated within the RISC-V + CGRA system. MMIO hardware accelerators for CRYSTALS-Kyber (now ML-KEM), Falcon and ASCON under development at UPM-CEI (externally, not within MYRTUS) could be potentially integrated in the future.
<i>Data management</i>	Being an edge node, most of the data is stored locally in main memory. Ongoing integration between the RISC-V + CGRA nodes and the DKB via standard APIs.
<i>Resource management Orchestration</i>	Computing offloading models for both CGRA and ARTiCo ³ [ARTiCo3-2025] architectures. Standard GNU/Linux distribution (with custom kernel) available. Integration of lightweight Kubernetes (K3s) and Ligo services for multi-cluster execution.
<i>Network</i>	Standard TCP/IP network stack on each node via GNU/Linux. Potential extension to MQTT and other network protocols.
<i>Monitoring and Observability</i>	Run-time monitoring infrastructure to acquire online power consumption and computing performance (#cycles) traces. Lightweight version of the data to be made available via DKB.
<i>Artificial Intelligence</i>	Node-level data-driven management strategies from Pillar 2 to select, at run time and based on the traces acquired by the monitoring subsystem, the best accelerator configuration. Deployment of AI kernels on the CGRA via the automated MLIR-based compilation tools from Pillar 3.

5.1.2 Fog components

5.1.2.1 Fog Micro Data Centres

The Fog Micro Data Centre (FMDC), developed by HIRO, represents a modular and easily scalable data centre solution. FMDC can range in size from small enclosures to larger container units, offering integrated power and cooling capabilities. A key characteristic of the FMDC is its ability to guarantee low to negligible processing latency, enhanced resilience through the avoidance of network losses, and improved security. This is achieved through its strategic placement in the fog layer of the MYRTUS architecture, acting as an intermediary point between edge devices and the cloud. The general description of the FMDC is provided in [Table 5-5](#).

Table 5-5 : Synopsis of the FMDC component.

Fog Micro Data Center [HIRO]			
ID: R4	Licence: N.A.*	Owner: HIRO	Contacts: myrtus-hiro@hiro-microdatacenters.nl



Short Description: FMDCs guarantee: low-to-no processing latency; improved resiliency (no network loss) and security; cost savings (due to incremental deployment and composability) and scalability support.	
Require	FMDC will require information from the different components at the edge.
Provide	FMDC collaborates with other EMDC's (East-West), 5G/6G and distant cloud (north-south) to provide optimal performance.
Input	Kubernetes deployable manifest and information from other components to be stored in DKB.
Output	The application is deployed at the fog layer with close proximity to the edge, so it will provide faster data exchange with minimal latency.
MYRTUS layer	Fog
TRL@M0	3
TRL@M36	5

The internal hardware architecture of the FMDC cluster is illustrated in [Figure 5-7](#). This cluster is supported with the Harvester [Harvester-2025] deployed in the FMDC itself which aims at managing virtual machines. Harvester and Rancher [Rancher-2025] together are the solutions adapted to support virtualization and manage **K8s clusters** at the FMDC level. For initial integration of all components with FMDC in MYRTUS continuum, a single **Kubernetes** cluster is configured, with a total of 8 CPUs, 16 GB RAM. Within the cluster there are 2 worker nodes configured with 2 CPUs 4GB RAM each and one master node with 4 CPUs and 8 GB RAM.

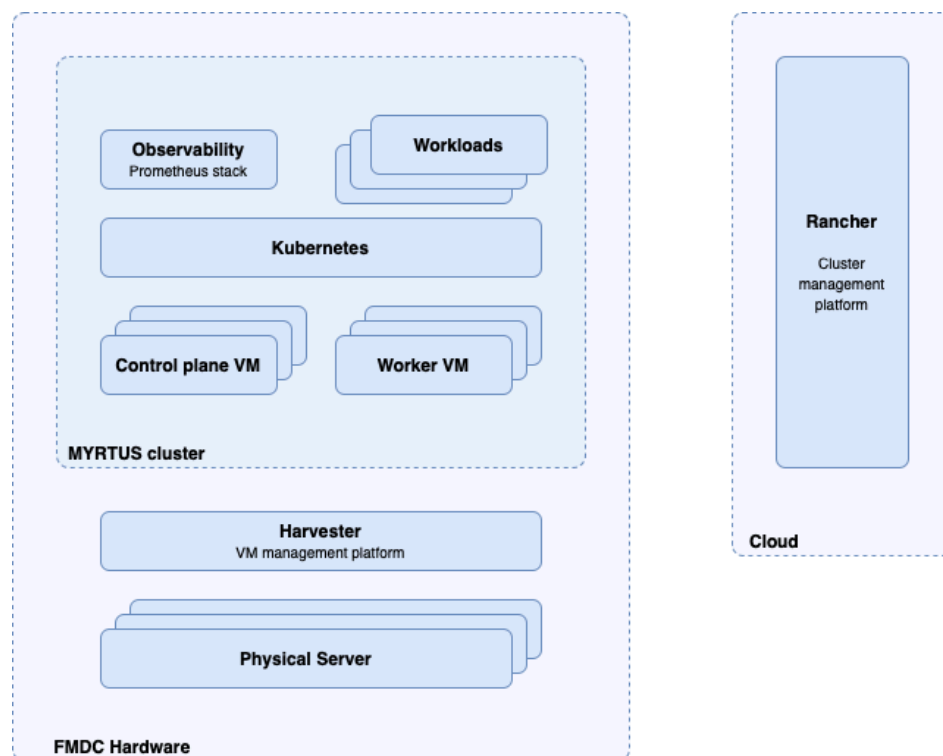


Figure 5-7: FMDC High Level Hardware Details.

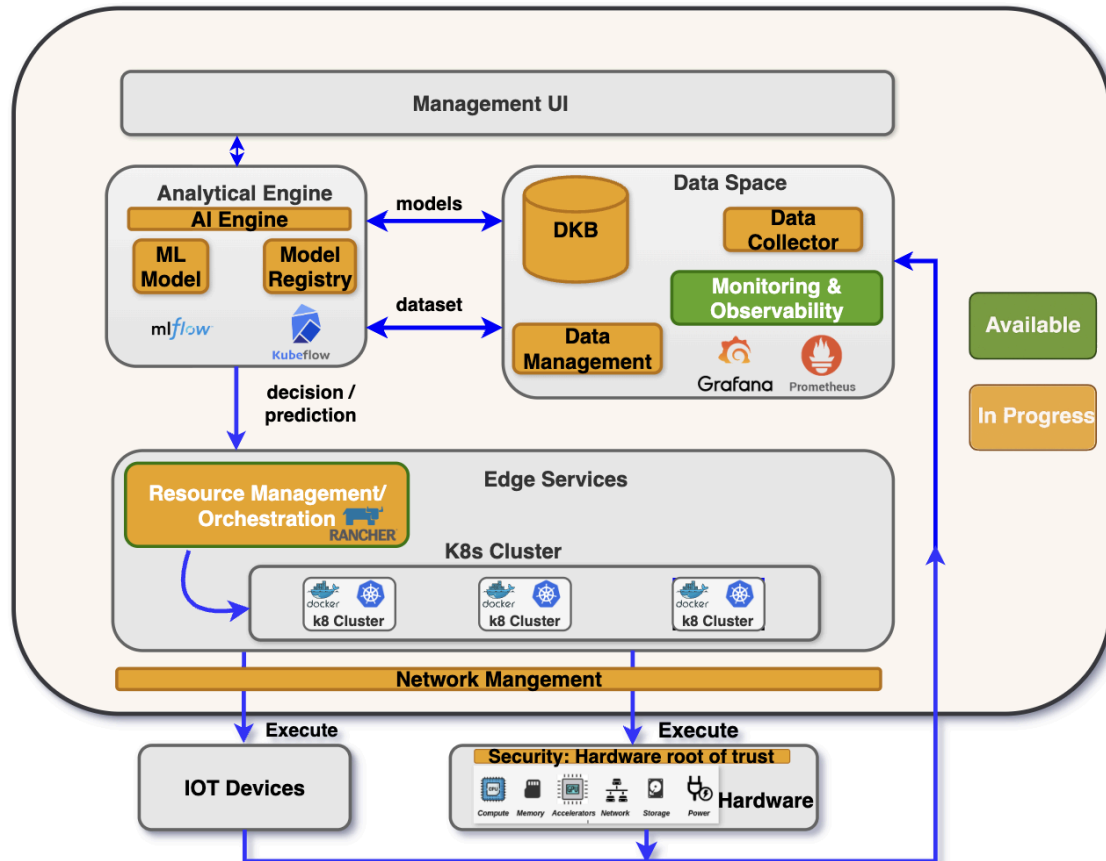


Figure 5-8: FMDC Architecture & EU-CEI Mapping. Blocks highlighted in green are the components already available, while the ones in yellow are under development.

The Internal FMDC architecture consists of three main components, i.e, analytical engine, dataspace and edge services as shown in [Figure 5-8](#).

- The **analytical engine** which is responsible to streamline the ML model lifecycle by providing personalized recommendations for models and datasets, enabling users to efficiently manage tracking, training, experimentation, validation, and serving within a single platform. The model registration is done through model registry and the life cycle management of models is taken care of by tools like mlflow and kubeflow.
- **Dataspace** provides a robust solution for managing and controlling data products within a decentralized and data-oriented architecture. The platform enables users to retain full control over their own data products by storing them locally while facilitating data access and requests through the platform. This approach ensures complete control over data access, usage, and governance provided by the data management. The DKB and data collector work together to manage datasets, enabling efficient data handling and replication. The monitoring & observability within dataspace is used to measure FMDC's metrics (such as cpu, memory utilisation,energy



etc.) and workload deployed at FMDC for real-time oversight of data-driven processes. It's handled using tools like Grafana and Prometheus.

- The **edge services** are responsible for resource management and orchestration that ensure proper allocation of computing resources across nodes within FMDC. The responsibility of the orchestrator is to optimize the workload within FMDC by selecting the most efficient node. The kubernetes cluster management is managed using tools like Rancher.
- The **network management** ensures seamless connectivity between IoT devices and the core FMDC architecture. Security mechanisms, including hardware-based root of trust, are integrated to protect the system against unauthorized access and potential threats. It is important to highlight the fact that the whole approach is aligned with the EU-CEI Mapping that can be witnessed in [Figure 5-8](#) and [Table 5-6](#).

The management unified graphical interface is used to manage all the management related activities of FMDC (such as checking the incoming requests, monitoring, managing clusters, workloads and resources).

Table 5-6: Mapping of HIRO FMDC to the EU-CEI building blocks.

EU-CEI Block	HIRO FMDC
<i>Security and Privacy Trust and Reputation</i>	• Support of Role-based access control (RBAC). • Compatibility with the Gaia-X Trust Framework.
<i>Data management</i>	• Distributed database and DKB support. • Data Replication using Minio.
<i>Resource management Orchestration</i>	• Managed standard K8s service. • Exploration of Multi-cluster support. • K8s management using Rancher.
<i>Network</i>	• Supports different communication protocols (REST, MQTT, UDP, TCP, NATS etc.).
<i>Monitoring and Observability</i>	• Evaluated metrics (such as cpu,memory,disk storage,file storage,network etc) to be monitored. • Monitoring the metrics using Grafana and Prometheus. • Exploring the possibility of energy metrics.
<i>Artificial Intelligence</i>	• The FMDC AI engine is a comprehensive platform for managing the entire AI model lifecycle with a recommendation engine for automated dataset and model. • Implemented ML life cycle phase (registration, training, validation, serving, and deployment). • Developing dataset registration capabilities. • Adding support for federated learning, model monitoring, and automation.

Here we discuss the results with respect to the plan described in D1.1 “UC Needs, requirements and KPIs v1” as a reference. When it comes to the observability and monitoring,



the assessment plan was to implement the integration of the observability stack that aims at presenting the state of various FMDC metrics, such as Memory usage, CPU etc. This goal was successfully achieved via providing the long list of the metrics that reflects the current state of the FMDC. [Table 5-7](#) provides the full list of metrics captured by the observability stack that includes earlier mentioned Prometheus, Grafana and InfluxDB.

Table 5-7: List of metrics that are monitored in the HIRO FMDC.

HIRO FMDC: Metrics
CPU: • CPU usage (%) per core • System load
RAM: • RAM usage (%) free • RAM usage (%) used
Memory: • Swap usage and availability • Space availability on the disc
Disk storage: • IOPS (Input/Output Operations per Second) • Disk Space Used (%)
Network • Tx and rx: speed (Mbps), Bytes, data packets • Latency (ms) • Jitter (latency variation) • Packet Dropped (%) • Number of open TCP/UDP connections

The assessment of the described developments, demonstrates the FMDC's contribution toward achieving the project's KPIs. It covers **KPI1.2** which is supported by AI driven approach at FMDC to improve overall efficiency by placing the most optimised nodes in the kubernetes cluster and **KPI1.3** is addressing the energy efficiency by offloading the workloads from fog to edge layer where FMDC at fog layer is energy optimised.

Overall these contributions are inline with MYRTUS objectives **T1.00.a**, in which FMDC supports standard communication interfaces (such as HTTP, MQTT, etc.) to allow seamless connectivity with the other components and having assessed Minio, MongoDB and ETCD as DKBs for flexible data storage capabilities. **T1.00.b** achieved using configurable FMDC by standard kubernetes-based architecture.

KPI1.1 and **T1.00.c** will be covered in WP4 by implementing Gaia-X in FMDC, by hosting verifiable credentials for the instances and running SHACL-based compliance checks with the shapes defined in [Section 4.3](#) to ensure Gaia-x Tier 1 level of trust.

5.1.2.2 SenseIQ

[Table 5-8](#) depicts a brief overview of the SenseIQ component from ABI [Fanni-2022] . ABI SenseIQ is a smart multi-sensor gateway based on the ABI Yocto-based Embedded Linux



Distribution (Ability). It includes an edge part with light computation capabilities and a cloud part interfaces with an operator.

Table 5-8: Synopsis of the SenseIQ component.

ABI SenseIQ – Abinsula smart multi-sensor gateway			
ID: R3	Licence: GNU GPL	Owner: ABI	Contacts: giuseppe.meloni@abinsula.com tiziana.fanni@abinsula.com katuscia.zedda@abinsula.com
Short Description: ABI SenseIQ provides a unified management system, contributing to the secure and safe integration of a cooperative embedded system in a system-of-systems.			
Require	Definition of the data format and of the protocol for the components in the system.		
Provide	A unified management system for heterogeneous components.		
Input	ABI SenseIQ processes inputs from heterogeneous components in the system.		
Output	ABI SenseIQ provides output to heterogeneous components in the system		
MYRTUS layer	Fog		
TRL@M0	4-5		
TRL@M36	6		

The ABI SenseIQ component is placed at the fog layer and is meant to transparently manage communication among the elements of edge components of the continuum on one side and with the cloud on the other side (see [Figure 5-9](#)).

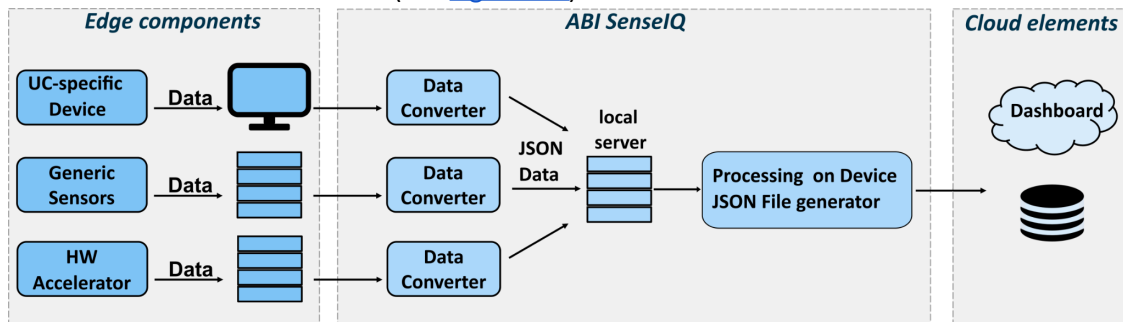


Figure 5-9: schematic overview of the ABI SenseIQ Multi-Sensor Gateway.

On the edge side, it supports different communication protocols (REST, MQTT, UDP, TCP, NATS e gRPC.) and accepts inputs described with custom-interfaces (data-interchange format). For each interface, a data converter module normalises the data interface, producing a standard JSON. Locally, ABI SenseIQ processes the data on device, and produces a standard JSON (or the necessary agreed data-interchange format) that can be sent to the cloud elements. Main characteristics of ABI SenseIQ are

- no vendor lock-in,
- not application specific,
- generic and modular component,
- main communication protocols, and
- user-defined interfaces.



The strategy behind the evolution of the ABI SenseIQ is based on: 1) extend the communication capabilities (integrating new protocols) and 2) improve the processing capabilities (integrating a decision node in the gateway to trigger system reconfiguration and supporting the computation near the sensor) in order to make it able to support decision making and trigger (sub)-system reconfiguration. In MYRTUS, ABI SenseIQ is extended to

- provide Kubernetes support (e.g., K3s) for resource management and orchestration,
- support different security levels, guaranteeing security and privacy as well as trust and reputation, and
- include a monitoring layer, for monitoring and observability.

The [Table 5-9](#) depicted the envisioned mapping with the EU-CEI building blocks.

Table 5-9: Mapping of ABI SenseIQ to the EU-CEI building blocks.

EU-CEI Block	ABI SenseIQ
<i>Security and Privacy Trust and Reputation</i>	The ABI SenseIQ component already integrates a lightweight cryptography module that allows for secure communication based on AES encryption/decryption. In MYRTUS, ABI SenseIQ will be extended to support different security levels according to MYRTUS security specifications. Furthermore, it will be extended to provide implementations aligned with the Gaia-X pillars.
<i>Data management</i>	ABI SenseIQ is not only a Hub for data exchange, but also a component with processing capabilities. In MYRTUS its processing capabilities are extended and the connection with different databases is being explored.
<i>Resource management and Orchestration</i>	The ABI SenseIQ component already offers the support of a light OS (Ability, the ABI Yocto-based Embedded Linux Distribution). In MYRTUS it is extended to provide K3s support.
<i>Network</i>	ABI SenseIQ supports different communication protocols (REST, MQTT, UDP, TCP, NATS e gRPC, etc.) and accepts inputs described with custom-interfaces (data-interchange format). In MYRTUS it is extended according to the protocols and interfaces defined for the MYRTUS Reference Infrastructure and to ensure connection with other components in the continuum.
<i>Monitoring and Observability</i>	A list of metrics has already been defined (see Table 5-10). Furthermore, we are evaluating the usage of Prometheus and Grafana into a Kubernetes cluster for monitoring and observability and the usage of openTelemetry for metrics management and testing.

Building on the foundations outlined above, significant progress has been made in extending the gateway's functionalities, contributing to achieving **MYRTUS objective T1.00.a** and **T1.00.b**. A major advancement has been the integration of Kubernetes support, achieved through the deployment of a lightweight K3s cluster. Within this environment, we have established a cluster architecture consisting of two primary nodes:



- One node hosting a minimal Python application designed to interact with a MongoDB database by performing both write and read operations.
- Another node running MongoDB, ensuring a reliable and scalable data storage solution inside ABI SenseIQ itself.

To facilitate internal development and streamline deployment procedures, comprehensive documentation has been created. This documentation includes detailed, step-by-step instructions for setting up a local K3s cluster on Ubuntu, deploying MongoDB alongside a Python-based application, and establishing a connection to MongoDB using MongoDB Compass.

In addition to the MongoDB-based setup, a parallel implementation has been developed, featuring a node that runs an application designed to interact with an external ETCD database. This provides an alternative data storage mechanism, enhancing the gateway's flexibility in different deployment scenarios.

Furthermore, an additional management node has been introduced to oversee cluster operations. This node is responsible for managing the execution of services, including starting and stopping nodes dynamically in response to incoming requests received via REST API. This management layer adds a crucial automation component, enabling more efficient resource utilization and system adaptability.

To enhance observability and system monitoring, we have also integrated Prometheus and Grafana within the K3s cluster. These tools enable real-time performance tracking, metric collection, and visualization of key system parameters. A preliminary list of the metrics that can be monitored is presented in [Table 5-10](#).

Table 5-10: Preliminary list of metrics to be monitored in the ABI SenseIQ Multi-Sensor Gateway.

ABI SenseIQ: Metrics to be monitored	
CPU:	
• CPU usage (%) per core, frequency (MHz), temperature (°C) • RAM usage (%) e free • Swap usage and availability • Space availability on the disc • IOPS (Input/Output Operations per Second) • Temperatura of the SoC • Total energy consumption • Tx and rx: speed (Mbps), Bytes, data packets • Latency (ms) • Jitter (latency variation) • Packet loss (%) • Number of open TCP/UDP connections • Active processes and resource consumption • Blocked or Zombi processes	
System Log:	
• Critical errors or warnings (dmesg, syslog) • Security events (tentative of non authorized access)	
Uptime:	



- Time since last reboot
- Number of active containers
- pod/container status(running, crashloop, etc.)
- Resources used by the containers

Finally, the ABI SenseIQ has been implemented on a Custom HW with NXP iMX8 SOM.

In parallel, activities necessary for a first integration of the gateway with the edge devices have been carried out. ABI SenseIQ has been extended to support HTTP protocol to provide compatibility with the work described in [Section 5.2.2](#), thus adding HTTP to the supported protocols, and the preliminary integration work demonstrates the correctness of bidirectional communication. This integration is a first step toward the achievement of MYRTUS **KPI 1.1** and **KPI 1.3**. This work is aligned with the plans defined in deliverable D1.1 “UC Needs, requirements and KPIs v1”, which envisioned that by M18 the ABI SenseIQ multi-sensor gateway would have been connected with at least the heterogeneous FPGA-based edge node at the edge, to implement efficient and seamless edge-to-fog exchanges (and vice versa).

5.1.3 Components implemented in the Healthcare Use Case - Smart garment.

UNICA developed the first prototype of the custom smart garment measuring the electromyographic signals and embedding vibroactuators. These sensing and actuating components will be connected to the MYRTUS edge layer that will handle all the related processing. In the next months, the design of the garment will be finalized, along with all the functional tests.

The smart garment is a long-sleeved, tight and stretchable synthetic shirt, bilaterally functionalized with custom textile electrodes (both for left and right arm), described below. The electromyographic (EMG) signals of the deltoid (lateral head), biceps brachii and triceps brachii muscles will be acquired in real time at 2kHz using two commercial wearable Due+Lite probes by OT Bioelettronica [DuePlus-2025], positioned on the arm and attached through velcro straps sewn on the same garment. The vibrotactile feedback will be implemented by a MetaMotionRL+ actuator [MetaMotionRLPlus-2025], positioned at the wrist level. Such a wearable device offers real-time and continuous monitoring of motion and environmental sensor data, featuring a coin vibration motor to provide vibrotactile feedback.

The textile electrodes adopted for the MYRTUS project are based on PEDOT:PSS (Clevios™ PH1000, produced by Heraeus, Hanau, Germany) [AELM-2015], a conductive polymer commonly used in bioelectronic applications due to its biocompatibility and stability. Despite these favorable properties, PH1000 exhibits relatively limited conductivity, which can be a limitation in applications that require high-conductivity electrodes, such as biopotential monitoring. For this reason, we explored specific modifications/functionalizations to the commercial PEDOT:PSS formulation through the introduction of additives aimed at improving its conductivity. In particular, the effects of ethylene glycol (EG) and deep eutectic solvents (DESs). EG is known to improve the conductivity of PEDOT:PSS by modifying the polymer matrix morphology, leading to more efficient charge transport within the polymer itself. On the other hand, the use of DESs can enhance the ionic conductivity of the polymer, reducing the electrode-skin interface impedance.

Within the MYRTUS project, the combined impact of EG and DES on the electrical properties



of PEDOT:PSS is being investigated, with particular focus on improving its performance for biomedical applications in the context of textile electrodes for biopotential monitoring. To assess the impact of the proposed functionalization, the following formulations were compared:

- (I) Pristine PH1000: the commercial formulation, used as the baseline reference;
- (II) PH1000 + EG + DES.

Both screen printing and spray coating techniques have been used as the main methods to apply the PEDOT:PSS-based functionalized inks onto fabrics (see [Figure 5-10](#)).

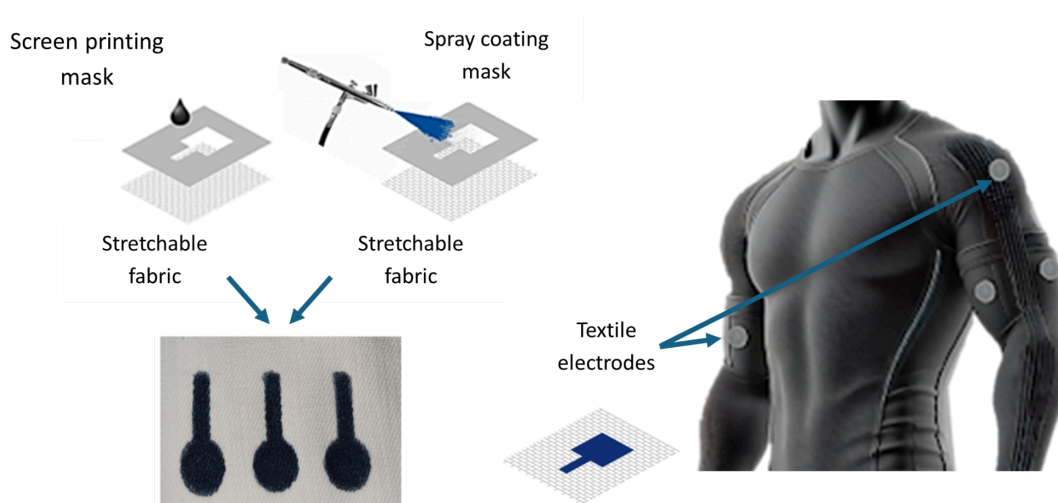


Figure 5-10: Schematic of the fabrication process of textile electrodes via screen printing and spray coating, and their integration onto the final smart garment.

Screen printing is a widely used technique for creating conductive layers on flexible substrates. It involves transferring ink onto a substrate, such as fabric, using a stencil or mask to create the desired pattern. The screen printing technique enables high reproducibility manufacturing, which can be easily adapted to various types of fabrics. The thickness of the deposited layer can be controlled by adjusting the ink viscosity and the number of printing passes, making it ideal for producing uniform conductive material layers. This method is particularly suitable for large-scale production of flexible and adaptable electrodes, which require consistent quality across all devices.

Spray coating, on the other hand, involves the atomization of an ink, which is then sprayed onto a target substrate (in this case, a fabric). Spray coating is particularly advantageous for ensuring uniform deposition of layers on large or irregular surfaces, and for achieving fine control over the thickness of the conductive layer. The spray process can be controlled by varying parameters such as distance, pressure, and nozzle size.

Both techniques are highly compatible with textile substrates, ensuring that the final electrodes retain the characteristics of the original fabrics, which is crucial for wearable applications. Furthermore, these methods allow for scalable production, making them suitable for large-scale fabrication of functional and flexible textile electrodes.



The textile electrodes, fabricated on cotton using the formulations previously described, were preliminarily characterized as shown in [Figure 5-11](#).

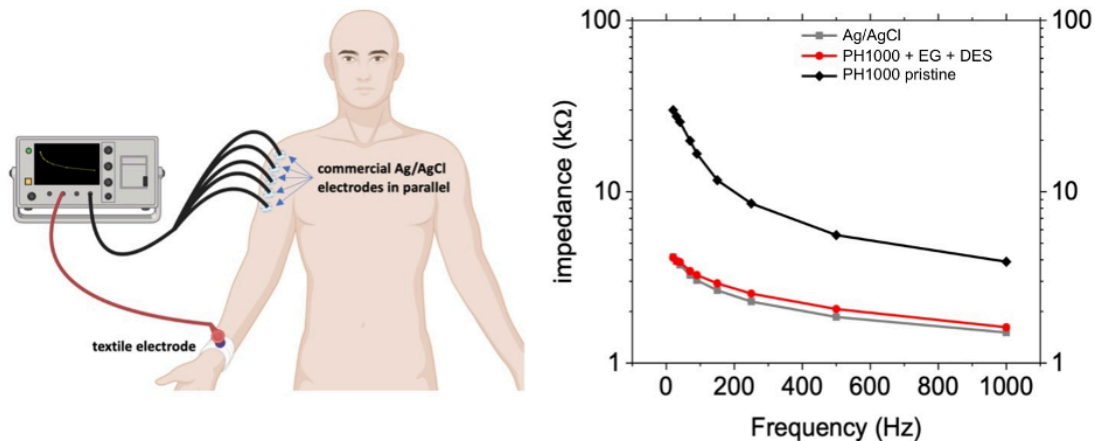


Figure 5-11: Setup for electrode-skin impedance measurement and obtained results.

The measurement was conducted using a LCR impedance meter (4284A, Agilent Inc.). Impedance was measured at various frequencies (20 Hz, 30 Hz, 40 Hz, 70 Hz, 90 Hz, 150 Hz, 250 Hz, 500 Hz, 1 kHz) to evaluate the electrode's behavior comprehensively.

Prior to characterization, a few drops of NaCl saline solution were deposited on the electrode to minimize unwanted interface effects between the electrode and the skin. The saline solution simulates the presence of sweat, a common condition in real-world applications, as hypothesized in previous works on PEDOT:PSS-based textile electrodes.

As shown in [Figure 5-11](#), the electrode fabricated with PH1000 + EG + DES ink exhibited electrical performance comparable to that of commercial pre-gelled electrodes across all tested frequencies, with impedance values decreasing with frequency, as expected.

It is important to note that this is a preliminary characterization, conducted using a fabric slightly different from the one that will be used for the final smart garment prototype. However, the results are promising, especially when comparing the impedances of the non-functionalized textile electrode, which shows approximately one order of magnitude higher values at low frequencies.

5.1.4 Components implemented in the Mobility Use Case

In the context of the MYRTUS Mobility UC, a set of five network cameras (Axis Q1656-LE) have been Installed within two adjacent intersections in the city of Helmond. They are constituting the MYRTUS edge layer of the use case.

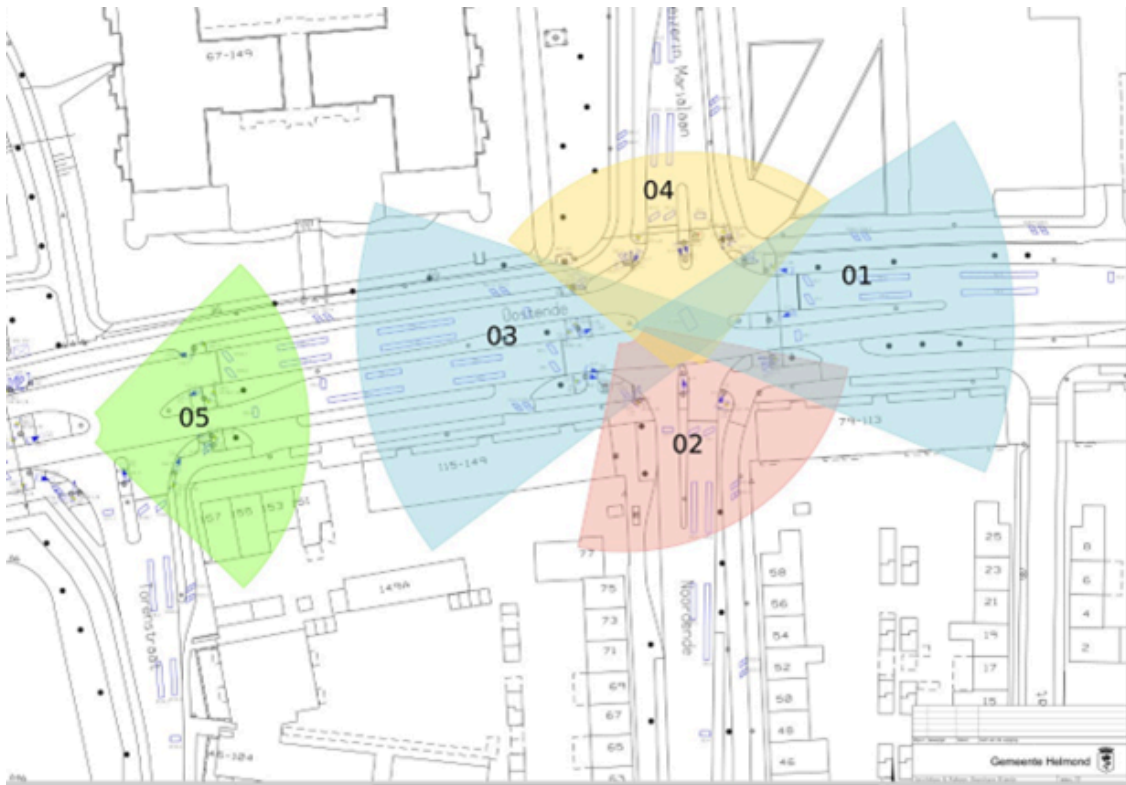


Figure 5-12: Use case Intersections areas covered by the cameras.

Axis Q1656-LE [AXIS-Q1656-LE-2025] camera is an outdoor IP camera. Based on the Axis system-on-chip (SoC) ARTPEC9, it integrates a Deep Learning Processing Unit (DLPU), and offers support for advanced features and powerful applications based on deep learning on the edge. It also offers applicative feature extension capacity, leveraging the Axis Camera Application Platform (ACAP) SDK. This last enables the Docker runtime to be used on the camera, enhancing the device capacity to be integrated into MYRTUS infrastructure as an edge device, capable of dynamically modifying its processing chain using containerized workloads.

This is a low resource device, based on aarch64 architecture, but it allows embedded deep learning based detection at the edge. This brings the advantages of low power, low latency, and privacy friendly to video processing based applications.

In order to extend the camera toward an edge device suitable for the use case, the following tasks are planned:

- leverage Axis Docker ACAP feature to integrate Mobility UC processing functions as Docker containerized software workloads.
- Collect system and applicative performance metrics
- Integrate cameras within a Kubernetes cluster, for resource management and orchestration (using K3s for instance).
- Interface with other MYRTUS components involved in the UC (MIRTO agent, DKB for instance)



Please note that, these cameras, having embedded computational capabilities and being able to run MIRTO agents, are to be considered as part of the multi-layer computing continuum infrastructure even though representing not generic computing nodes, but rather an application-specific legacy devices. This proves also that MYRTUS Reference Infrastructure is interoperable also with commercial and proprietary devices.

As a current status:

- A first release of the road user tracker functionality of the use case has been adapted to run embedded in the camera as a Docker container. This module outputs a sequence of detected road users' metadata, formatted according to an Intelligent Transport System (ITS) Collaborative Perception Message (CPM) payload. It can be remotely activated and deactivated using standard Docker API.
- A Docker container hosting a Prometheus node_exporter script has been built for the camera, and tested on the device, making camera system metrics available to a fog or cloud hosted Prometheus like instance.
- Integration within a Kubernetes cluster is under study. K3s compliant Kubernetes distribution seems to be a good candidate, as it is lightweight and well adapted to aarch64 devices.

5.2 Integration

5.2.1 Edge to Edge

The edge layer consists of heterogeneous components that might not be directly accessible to each other due to being physically placed in geographically remote locations (they will be close to sensors producing input data) or restricted by the computing network they are part of. The purpose of this edge-to-edge integration is to avoid these restrictions by joining the edge components into a common infrastructure to enable seamless horizontal orchestration at the edge layer.

As described in previous sections, the edge components representing novel computing nodes provided within MYRTUS, UPM and UNICA/UNISS components, consists of a programmable device (FPGA) to perform hardware acceleration and an ARM processor (for this deployment, UPM is relying on the ARM + ARTiCo³ nodes [ARTiCo³-2025] , as that node flavor is currently more mature than the RISC-V + CGRA one; in later stages of the project both solutions will coexist) with a custom Linux-based operating system that manages the component operation. Among the processes running on the operating system is K3s, a lightweight container orchestration solution that enables: (1) the creation of local clusters made of components that are already accessible to each other, and (2) the orchestration of containerized workloads into those local clusters.

The edge-to-edge integration presented in this section leverages Ligo to bridge remote and independent K3s clusters, creating a multi-cluster network topology that combines the computing resources of those remote K3s clusters. The result is the abstraction of edge components into a unique K3s cluster where every edge component is accessible to each other, effectively enabling seamless horizontal orchestration of the workloads while the underlying infrastructure remains hidden.



A preliminary implementation of this integration is illustrated in [Figure 5-13](#). The base infrastructure is composed of a set of edge components from UPM treated as a K3s cluster located at the UPM facilities in Madrid (Spain) and a set of edge components from UNICA also treated as a K3s cluster located at UNICA laboratories in Cagliari (Italy). This preliminary implementation leverages Liko to combine these two remote K3s clusters into a unique K3s cluster. Note that the components in one cluster could not be directly accessed by the components in the other cluster without the peering features provided by Liko.

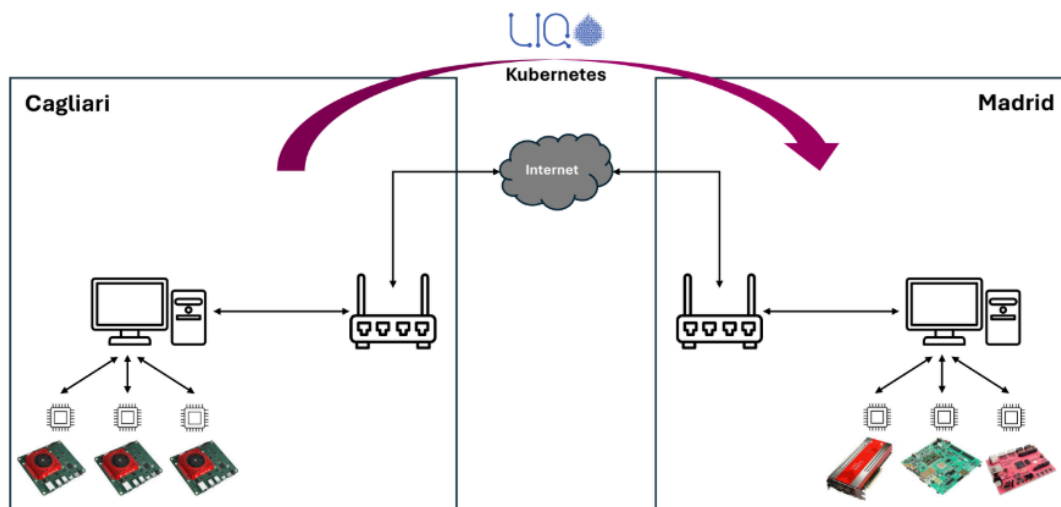


Figure 5-13: Schematic overview of the edge to edge integration of heterogeneous computing resources leveraging Kubernetes and Liko support.

To do so, a dedicated Liko process has been deployed in the master node of each remote K3s cluster. Afterwards, based on configuration files providing information about each remote K3s cluster (e.g., public IP address), Liko creates a peering between both remote clusters, resulting in a unique K3s cluster that combines their computing resources. Once this peering is available, containerized hardware acceleration can be deployed in any edge component. This current implementation has been successfully validated by deploying basic hardware acceleration applications on the edge components at UPM facilities in Madrid and UNICA labs in Cagliari following the general K3s deployment flow (as if both remote clusters were a unique K3s cluster). Therefore, the seamless horizontal orchestration capability and the scalability of the implemented edge-edge integration are demonstrated.

5.2.2 Edge to Fog

The preliminary integration of the fog-edge layers, illustrated in [Figure 5-14](#), aimed at establishing a seamless interaction between heterogeneous computing resources while optimizing performance and adaptability across different operational scenarios. At the fog layer, the ABI SenseIQ gateway serves as a critical intermediary in the infrastructure, providing processing capabilities and coordinating interactions between cloud and edge components. At the edge layer, the target platform was the UNICA/UNISS component, designed around ARM processors coupled with programmable logic (FPGAs). This hybrid architecture supports both operating system-level orchestration and dynamic hardware acceleration, making it well-suited for computationally intensive tasks.

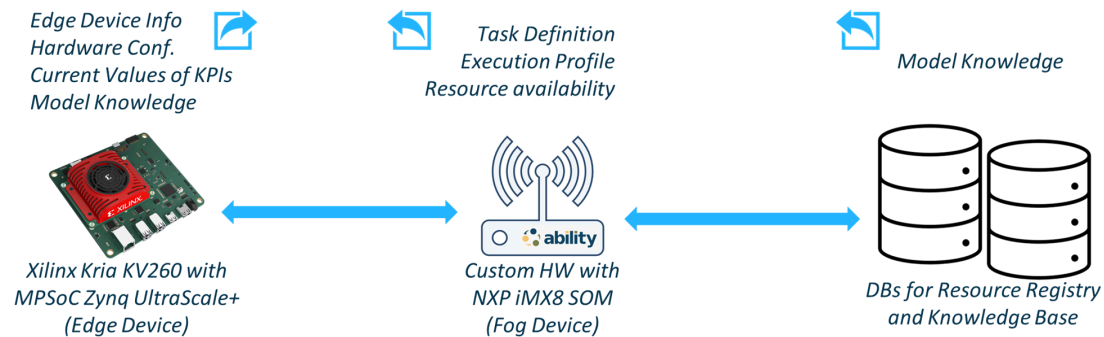


Figure 5-14: Schematic overview of the preliminary fog-edge integration.

As already explained in previous Sections, both the ABI SenseIQ gateway and the UNICA/UNISS component are integrated with a lightweight, customizable embedded OS, based on a custom embedded Linux distribution developed using the YOCTO ecosystem. In the case of the edge component, this tailored distribution was optimized for edge computing applications, ensuring minimal resource overhead while maintaining robust support for containerized workloads via K3s. Additionally, a comprehensive monitoring layer (based on an existing Python library) was implemented, allowing real-time tracking of system performance, resource utilization, and power consumption.

The core of this integration was the development of APIs to facilitate database interactions, enabling efficient state monitoring and adaptive control mechanisms for edge devices. Since some developments of the ABI SenseIQ gateway were still ongoing, a Mockoon server was deployed to emulate the ABI SenseIQ gateway, simulating providing the connectivity to non-relational databases and enabling real-time data exchange with the edge layer.

The APIs were designed with two key objectives: (1) to collect and analyze non-functional metrics, such as hardware resource usage and energy consumption, enabling informed decision-making and system optimization; and (2) to dynamically reconfigure FPGA-based hardware acceleration, adjusting computational resources based on predefined execution directives. Specifically, these APIs have been developed in Python based on the design principles of the REST architectural style, which exploits the HTTP protocol to perform CRUD operations on resources.

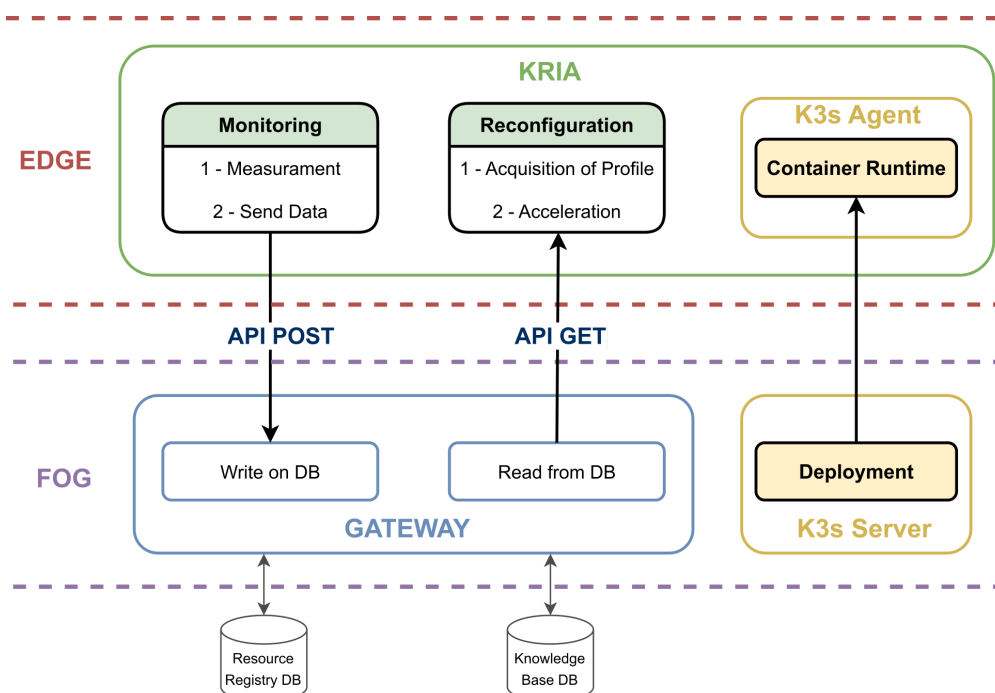


Figure 5-15: Schematic overview of the edge-fog implemented communication.

As illustrated in [Figure 5-15](#), currently, monitored data are communicated via specific APIs in order to be accessed in one of two specific databases (for resource registry and Knowledge Base) in the MYRTUS infrastructure. Based on this information, MYRTUS infrastructure management provides container execution directives associated with an execution profile. For this reason, a second application, also running continuously as an operating system service, retrieves via API the directives from the profiling database (the one in the Knowledge Base) and stores the current configuration in a log file, within the file system. Since the edge device is part of a Kubernetes cluster, it can handle the request to run a container containing a hardware-accelerated application. The latter will run in the established manner, following the profile indicated in the log file and possibly reconfiguring the hardware accelerator present on the programmable logic of the FPGA.



6 Conclusion

WP3, and Pillar 1 development, is progressing as expected.

Originally three different OOs were planned. These OOs and how the development in this first half of the project are contributing to reaching them are summarized in the following tables. All OOs, in particular the first two, are covered and are progressing. With respect to Gaia-X we are dependent on the Gaia-X advancements per se. In this first project interaction we put the basis to address also this OO, but more work at the infrastructure and component level is expected for the second iteration of the project.

T1.OO.a - Provide a layered reference infrastructure representing the skeleton of MYRTUS compliant systems, capable of ubiquitous data acquisition and distributed intelligent decision-making.

A layered reference infrastructure has been already defined (see [Figure 3-2](#)). It is composed of a diversity of computing nodes capable of different storing and processing capabilities both at the edge and at the fog, and different constraints have been defined to enable the assessment of trust before service federation and be sure about the identity, compliance status, and integrity of service providers.

At the edge, we are contributing with two novel computing nodes: the HMPSoCs FPGA-based accelerators and adaptive RISC-V processors with custom computing units. At the fog, we are contributing with two novel computing nodes: FMDCs and SenseIQ. FMDCs and SenseIQ support standard communication interfaces (such as HTTP, MQTT, etc.) to allow seamless connectivity among components.

Kubernetes (using the lightweight K3s version at the edge and also on SenseIQ and the K8s in the other computing nodes) and Liko support in all compute nodes are enabling the creation of a continuum of computing resources.

Distributed monitoring and observability (using tools such as Prometheus and Grafana at the fog and having assessed MongoDB and ETCD as DKBs for flexible data storage) capabilities have been already put in place to enable gathering the status of the whole infrastructure. Edge and fog nodes are embedding a Linux-based OS which facilitates data acquisition and management.

Partial integration of the envisioned continuum is already there, while the interface between the MYRTUS Reference Infrastructure and MIRTO cognitive engine architecture has been defined, but its implementation is currently in progress to allow distributed decision-making capabilities.

T1.OO.b - Provide customizable, adaptive and efficient fog/edge nodes: including adaptable computing devices, small, energy efficient micro datacentres, and modular gateway, to bridge edge and far cloud.

All edge and fog developed computing nodes are modular and customizable. Adaptivity at the edge is widely supported in all the novel edge resources developed within MYRTUS. The HMPSoC CGR accelerators leverage approximate computing techniques to offer different working points (trading-off for example energy efficiency for QoS), while the introduction of multithreading support further expands computational capabilities and delivers acceleration with respect to general-purpose computing. Additionally, on the RISC-V side, the integration of the CGRA accelerator within a CORE-V CVA6 SoC under GNU/Linux sets up the first step towards the automatic HW/SW partitioning and dynamic offloading at run time, offering a novel adaptive computing fabric. At the fog level both



developed components feature a modular Kubernetes-based architecture, supporting extensible and customizable interfaces.

Energy efficiency is optimized by design: hardware specialization is exploited in the case of HMPSoCs FPGA-based accelerators and adaptive RISC-V, while the FMDC is integrating cores with a good power usage effectiveness. Moreover, In addition to being an energy-efficient data center, FMDC also leverages an internal integrated AI-support to optimize workload placement for enhanced energy efficiency.

T1.00.c - Ensure compatibility with the Gaia-X Trust Framework for MYRTUS nodes (following the guidelines defined by the Gaia-X digital clearing house).

Gaia-X Trust Framework recommendations and guidelines have been studied. The definition of a set of operative requirements and strategies to align Pillar 1 technologies, in particular the cloud interface, with the Gaia-X Trust Framework is in progress. Currently, constraints and SHACL shapes that can be used by MIRTO Agents when invoking a SHACL engine have been designed. They can be used to support automated validation of verifiable credentials, ensuring that the MYRTUS-compliant computing continuum infrastructure will comply with the Gaia-X Trust Framework (Tier 1 Trust Level). How to implement the different strategies at component level is something to be refined in the second part of the project.

Three different KPIs were planned. These KPIs and how the development in this first half of the project are contributing to reaching them are summarized in the following tables. The KPI analysis demonstrates that, even if developments are still in progress and some numbers are yet to be collected, a positive trend is there.

KPI1.1 - MYRTUS Reference Infrastructure will be capable of integrating at least 2 cloud regions and 3/4 edge/fog environments composed of heterogeneous devices (e.g., CPU, GPU, FPGA, etc.), guaranteeing 100% availability of services for edge devices and near real time interactivity (<5ms) leveraging on the fog.

Activities necessary for a first integration of 3 heterogeneous FPGA-based edge nodes environments (HMPSoC CGR accelerators, CGRA accelerator + CVA6 RISC-V processor, ARM + ARTiCo3 structures) together and of the HMPSoC CGR accelerators with the ABI SenselQ have been carried out (see [Section 5.2.1](#) and [Section 5.2.2](#)). All nodes leverage GNU/Linux support, and both K3s and Liqo. The preliminary integration tests demonstrate the correctness of bidirectional communication, even though metrics in terms of availability and real-time interactivity have not been extracted yet.

KPI1.2 - One order of magnitude performance improvement using dedicated accelerators at the edge/fog layer.

At the edge a unified monitoring infrastructure is currently available, laying out the foundation for the practical evaluation of this KPI. Moreover, some lab tests on standalone kernels (as planned for this first half of the project) have been carried out. The FPGA-based HMPSoC CGR acceleration has been demonstrated on multi-thread implementation on AES, FFT and HEVC-filter, achieving between 7x and 40x speedup with respect to sw-only execution, depending on the application and the number of threads. The FPGA-based RISC-V processor with CGRA-based extensions has been tested on ReLU and 2D convolution benchmarks resulting in a preliminary speedup of 2.2x and 4.3x with



regards to their equivalent software implementation, respectively. In turn, the hardware acceleration platform based on ARTICo³ has been demonstrated in a block-based matrix multiplication application, achieving 33x speedup over software-based alternatives. Note that, the preliminary edge-fog integration activities already demonstrate the correctness of bidirectional communication.

KPI1.3 - 10-20% better energy efficiency when using heterogeneous HW/SW edge/fog computing solutions.

At the edge a unified monitoring infrastructure is currently available, laying out the foundation for the practical evaluation of this KPI. Moreover, some lab tests on standalone kernels (as planned for this first half of the project) have been carried out. The FPGA-based HMPSoC CGR accelerator has been used to realize CNN acceleration demonstrating to achieve inference with an energy budget of few tens of Watt (depending on precision), which is approximately one order of magnitude lower than the idle power consumption of the target HMPSoC when the OS is running (around 3W). The FPGA-based RISC-V processor with CGRA-based extensions and the hardware acceleration platform based on ARTICo³ have rendered approximately the same power consumption when accelerating applications as their software counterparts due to the high power consumption from the OS running on the host CPUs, and the high static power consumption of the FPGA devices. However, the performance improvements reported in KPI1.2 have led to significantly better energy efficiency values (proportional to the speedup). The common monitoring infrastructure developed in the first part of the project will be used to align the energy consumption measurements by considering both the static and dynamic dissipation produced by the CPUs and the FPGA on the HMPSoC. At the fog level on the FMDC Kepler has been identified to monitor the energy, but no actual tests on specific kernels have been run yet.



7 Glossary

Name	Description
Distributed Knowledge Base	A database containing blobs of information and/or files, that any component might need for persisting or sharing of knowledge.
Liqo	Liqo is an open-source project that enables Kubernetes clusters to share resources and workloads seamlessly. It allows for dynamic and secure multi-cluster environments, making it easier to manage and scale applications across different cloud providers or on-premises infrastructure.
MIRTO Agent	The core MIRTO cognitive engine (see D5.1) responsible for coordinating and accessing all MIRTO managers. Also the primary access point for the MIRTO software specialist for installing, monitoring and troubleshooting applications.
Resource Registry	A specialized interface on top of the DKB to disseminate information on the status of the MYRTUS infrastructure.

8 References

8.1 MYRTUS papers

- **[Palumbo-2025]** F. Palumbo, F. Ratto, C. Rubattu, M. K. Zedda, T. Fanni, V. Rao, B. Driessen, J. Castrillon. "Key Enabling Technologies for Cognitive Computing Continuum - MYRTUS Project Perspective". Accepted in the Design, Automation and Test in Europe Conference 2025 (DATE 25), Lyon, FR. **Available on Zenodo:** <https://zenodo.org/records/14609859> , [28/04/2025]
- **[Diez-de-Ulzurrun-2024]** I. D. De Ulzurrun, J. Encinas, A. Rodríguez and A. Otero, "Cloud-Edge Continuum Infrastructure for Reconfigurable Multi-Accelerator Systems," 2024 39th Conference on Design of Circuits and Integrated Systems (DCIS), Catania, Italy, 2024, pp. 1-6, doi: 10.1109/DCIS62603.2024.10769198 , [28/04/2025]
- **[Encinas-2025]** Encinas, J. et al. (2025). Leveraging Incremental Machine Learning for Reconfigurable Systems Modeling under Dynamic Workloads. ACM Trans. Reconfig. Technol. Syst. 18, 1 , [28/04/2025]
- **[Granja-2024]** J. Granja, "Integration of a CGRA Accelerator with a CVA6 RISC-V Core for the Cloud-edge Continuum," MSc Thesis, Universidad Politécnica de Madrid, 2024, <https://oa.upm.es/83488> , [28/04/2025]
- **[Manca-2024]** Manca, Federico, Francesco Ratto, and Francesca Palumbo. "Onnx-to-hardware design flow for adaptive neural-network inference on fpgas." International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation. Cham: Springer Nature Switzerland, 2024 , [28/04/2025]
- **[Thakur-2024]** S. Thakur, S. Banik, F. Regazzoni, "Energy Analysis of Cryptographic Algorithms in Server Environment". Proceedings of the 2024 on Cloud Computing



Security Workshop, (CCSW) 2024, Salt Lake City, UT, USA, October 14-18, 2024: 3-14 , [28/04/2025]

8.2 Other references

- **[Fanni-2022]** T. Fanni, G. Meloni, M. Melis, A. Solinas, M.K. Zedda, "The Multi-Sensor Gateway, a Unified Communication Scheme and Orchestration Actor for Heterogeneous Systems". In: Proceedings of the CPS Summer School PhD Workshop 2022, co-located with 4th Edition of the CPS Summer School (CPS 2022), Pula, Sardinia (Italy), September 19-23, 2022. Published in: CEUR Workshop Proceedings Vol-3252. ISSN 1613-0073. <http://ceur-ws.org/Vol-3252/paper3.pdf> , [28/04/2025]
- **[Shreshth-2019]** Shreshth Tuli, Redowan Mahmud, Shikhar Tuli, Rajkumar Buyya, FogBus: A Blockchain-based Lightweight Framework for Edge and Fog Computing, Journal of Systems and Software, Volume 154, 2019, Pages 22-36,ISSN 0164-1212 , [28/04/2025]
- **[Javed-2018]** A. Javed, K. Heljanko, A. Buda and K. Främling, "CEFIoT: A fault-tolerant IoT architecture for edge and cloud," *2018 IEEE 4th World Forum on Internet of Things (WF-IoT)*, Singapore, 2018, pp. 813-818 , [28/04/2025]
- **[Intel-2017]** Intel. "Seamless Edge-to-Cloud IoT Integration Speeds Time to Market." 2017.Intel,<https://www.intel.com/content/dam/www/public/us/en/documents/reference-architectures/seamless-edge-to-cloud-iot-reference-architecture.pdf> , [28/04/2025]
- **[OpenStack-2025]** OpenStack. "Cloud Edge Computing: Beyond the Data Center." *OpenStack*, <https://www.openstack.org/use-cases/edge-computing/cloud-edge-computing-beyond-the-data-center/> , [28/04/2025]
- **[Bonomi-2012]** F. Bonomi et al., 'Fog Computing and Its Role in the Internet of Things,' Proceedings of the MCC Workshop, 2012 , [28/04/2025]
- **[EU-CEI-2023]** Task Force 3: Architecture, "Developing a Reference Architecture for the Continuum - Concept, Taxonomy and Building Blocks," Oct. 2023. [Online]. Available: <https://doi.org/10.5281/zenodo.8403593> , [28/04/2025]
- **[Intel-IoT-2023]** Intel, ISeamless Edge-to-Cloud IoT Integration Speeds Time-to-Market. <https://www.intel.com/content/dam/www/public/us/en/documents/reference-architectures/seamless-edge-to-cloud-iot-reference-architecture.pdf> , [28/04/2025]
- **[Xu-2021]** X. Xu et al., 'Edge Intelligence: Architectures, Challenges and Applications,' IEEE IoT Journal, vol. 8, no. 4, pp. 2449–2462, 2021 , [28/04/2025]
- **[Encinas-2024]** J. Encinas, A. Rodríguez, A. Otero, and E. de la Torre, "Data-driven modeling of reconfigurable multi-accelerator systems under dynamic workloads," *Microprocessors and Microsystems*, vol. 107, p. 105050, 2024, doi: 10.1016/j.micpro.2024.105050 , [28/04/2025]
- **[Vazquez-2024]** D. Vazquez, J. Miranda, A. Rodriguez, A. Otero, P. D. Schiavone, and D. Atienza, "STRELA: STreaming ELastic CGRA Accelerator for Embedded Systems," *arXiv*, 2024, doi: 10.48550/arXiv.2404.12503 , [28/04/2025]
- **[Azure-IoT-2025]** [Microsoft], <https://azure.microsoft.com/en-us/products/iot-hub>, [25/04/2025]
- **[Aws-IoT-2025]** [Amazon],https://aws.amazon.com/iot/?nc1=h_ls, [28/04/2025]



- **[Google-IoT-2025]** [Google], <https://cloud.google.com/architecture/connected-devices/iot-platform-product-architecture>, [28/04/2025]
- **[Gaia-X-2025]** [Gaia-X Association], <https://gaia-x.eu>, [28/04/2025]
- **[IEEE-standard-2025]** [IEEE standard association], <https://standards.ieee.org/ieee/1934/7137/>, [28/04/2025]
- **[FogBus2-2025]** [University of Melbourne's CLOUDS Lab], <https://github.com/Cloudslab/FogBus2>, [28/04/2025]
- **[Eclipse-kura-2025]** [Eclipse Foundation], <https://eclipse.dev/kura/>, [28/04/2025]
- **[HiPEAC-2025]** [HiPEAC Network of Excellence], <https://www.hipeac.net/#/>, [28/04/2025]
- **[AIOTI-2025]** [Alliance for IoT and Edge Computing Innovation (AIOTI)], <https://aioti.eu/>, [28/04/2025]
- **[EU-CEI-IoT-2025]** [EUCloudEdgeIoT Initiative], <https://eucloudedgeiot.eu/>, [28/04/2025]
- **[Liqo-2025]** [ArubaKube], <https://liqo.io/>, [28/04/2025]
- **[Liqo-Peering-2025]** [ArubaKube], <https://docs.liqo.io/en/stable/features/peering.html>, [28/04/2025]
- **[Kubernetes-2025]** [Google], <https://kubernetes.io/>, [28/04/2025]
- **[Lightweight Kubernetes-2025]** [Google], <https://k3s.io/?ref=traefik.io>, [28/04/2025]
- **[Gaia-x-22.04-Release-2024]** [Gaia-x Association], <https://docs.gaia-x.eu/policy-rules-committee/trust-framework/22.04/>, [28/04/2025]
- **[Gaia-x-24.06-Release-2024]** [Gaia-x Association], <https://gaia-x.eu/wp-content/uploads/2024/09/Compliance-Documents-24.06.pdf>, [28/04/2025]
- **[Gaia-x-24.03-Release-2024]** [Gaia-x Association], <https://gaia-x.eu/wp-content/uploads/2024/05/An-Introduction-to-the-Gaia-X-Trust-Framework-2024-V4.pdf>, [28/04/2025]
- **[RDF4J-2025]** [Eclipse Foundation], <https://rdf4j.org/>, [28/04/2025]
- **[Stardog-2025]** [Stardog], <https://www.stardog.com/platform/features/data-quality-shacl/>, [28/04/2025]
- **[Gaia-X-TrustFramework-2025]** [Gaia-X Association], <https://registry.lab.gaia-x.eu/development/api/trusted-shape-registry/v1/shapes/jsonld/trustframework>, [28/04/2025]
- **[SHACL-NS-2025]** [W3C], <http://www.w3.org/ns/shacl#>, [28/04/2025]
- **[W3-VC-DATA-MODEL-2025]** W3C, <https://www.w3.org/TR/vc-data-model/>, [28/04/2025]
- **[SHACL-TR-2025]** W3C, <https://www.w3.org/TR/shacl/>, [28/04/2025]
- **[JSON-Web-Signatures-2025]** W3C, <https://www.w3.org/TR/vc-jws-2020/>, [28/04/2025]
- **[XMLSchema-2025]** [W3C], <http://www.w3.org/2001/XMLSchema#>, [28/04/2025]
- **[VerifiableCredentials-2018]** [W3C], <https://www.w3.org/2018/credentials#>, [28/04/2025]
- **[W3C-Security-2025]** [W3C], <https://w3id.org/security#>, [28/04/2025]
- **[ExampleShapes-2025]** [example.org], <https://example.org/shapes#>, [28/04/2025]



- **[EtcD-2025]** [Cloud Native Computing Foundation], <https://etcd.io/>, [28/04/2025]
- **[Redis-2025]** [Redis Ltd.], <https://redis.io/>, [28/04/2025]
- **[Cassandra-2025]** [The Apache Software Foundation], <https://cassandra.apache.org/>, [28/04/2025]
- **[MongoDB-2025]** [MongoDB Inc.], <https://www.mongodb.com/>, [28/04/2025]
- **[Minio-2025]** [MinIO, Inc.], <https://min.io/>, [28/04/2025]
- **[Cockroachdb-2025]** [Cockroach Labs], <https://www.cockroachlabs.com/>, [28/04/2025]
- **[InfluxDB-2025]** [InfluxData], <https://www.influxdata.com/>, [28/04/2025]
- **[TimeScale-2025]** [Timescale, Inc.], <https://www.timescale.com/>, [28/04/2025]
- **[MDC-Tool-2025]** [Mdc-suite], <https://github.com/mdc-suite/qonnx2mdc/tree/main>, [24/04/2025]
- **[CVA6-2025]** [OpenHW Group], <https://github.com/openhwgroup/cva6>, [28/04/2025]
- **[StrelaCVA6-2025]** [Des-cei], https://github.com/des-cei/strela_cva6, [28/04/2025]
- **[ARTICo³-2025]** [Des-Cei, CEI-UPM], <https://github.com/des-cei/artico3>, [24/04/2025]
- **[Harvester-2025]** <https://harvesterhci.io/>
- **[Rancher-2025]** [SUSE], <https://www.rancher.com>, [28/04/2025]
- **[OpenAPI-2025]** <https://swagger.io/specification/>
- **[DuePlus-2025]** [OT Bioelettronica], <https://otbioelettronica.it/due/>, [28/04/2025]
- **[MetaMotionRLPlus-2025]** [MbientLab Inc.], <https://mbientlab.com/store/metamotionrl-p/>, [28/04/2025]
- **[AELM-2015]** [Advanced Electronic Materials], <https://doi.org/10.1002/aelm.201500017>, [28/04/2025]
- **[AXIS-Q1656-LE-2025]** [Axis Communications AB], <https://www.axis.com/products/axis-q1656-le>, [28/04/2025]



9 Appendix

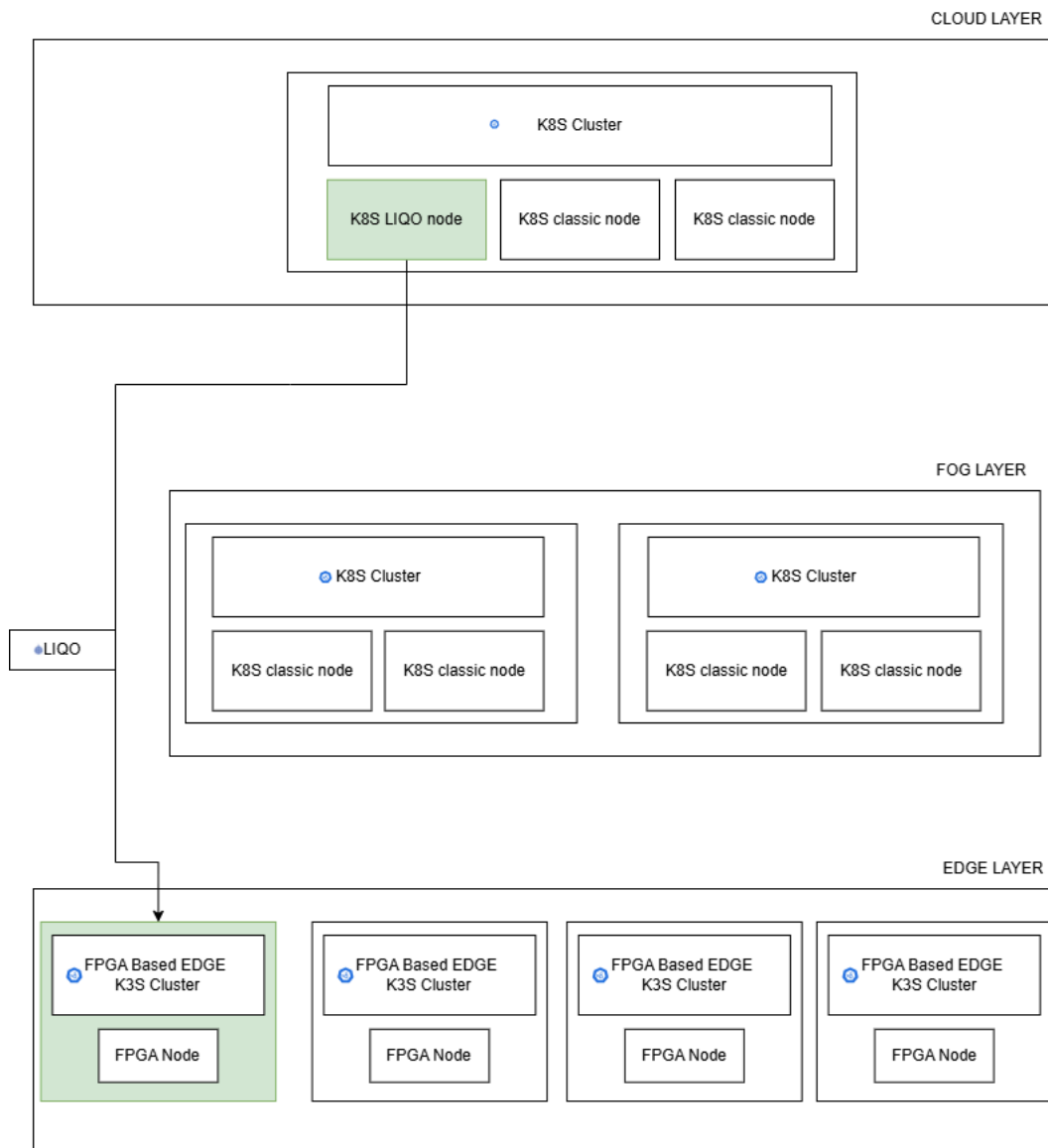


Figure 9-1: cloud to edge interaction.

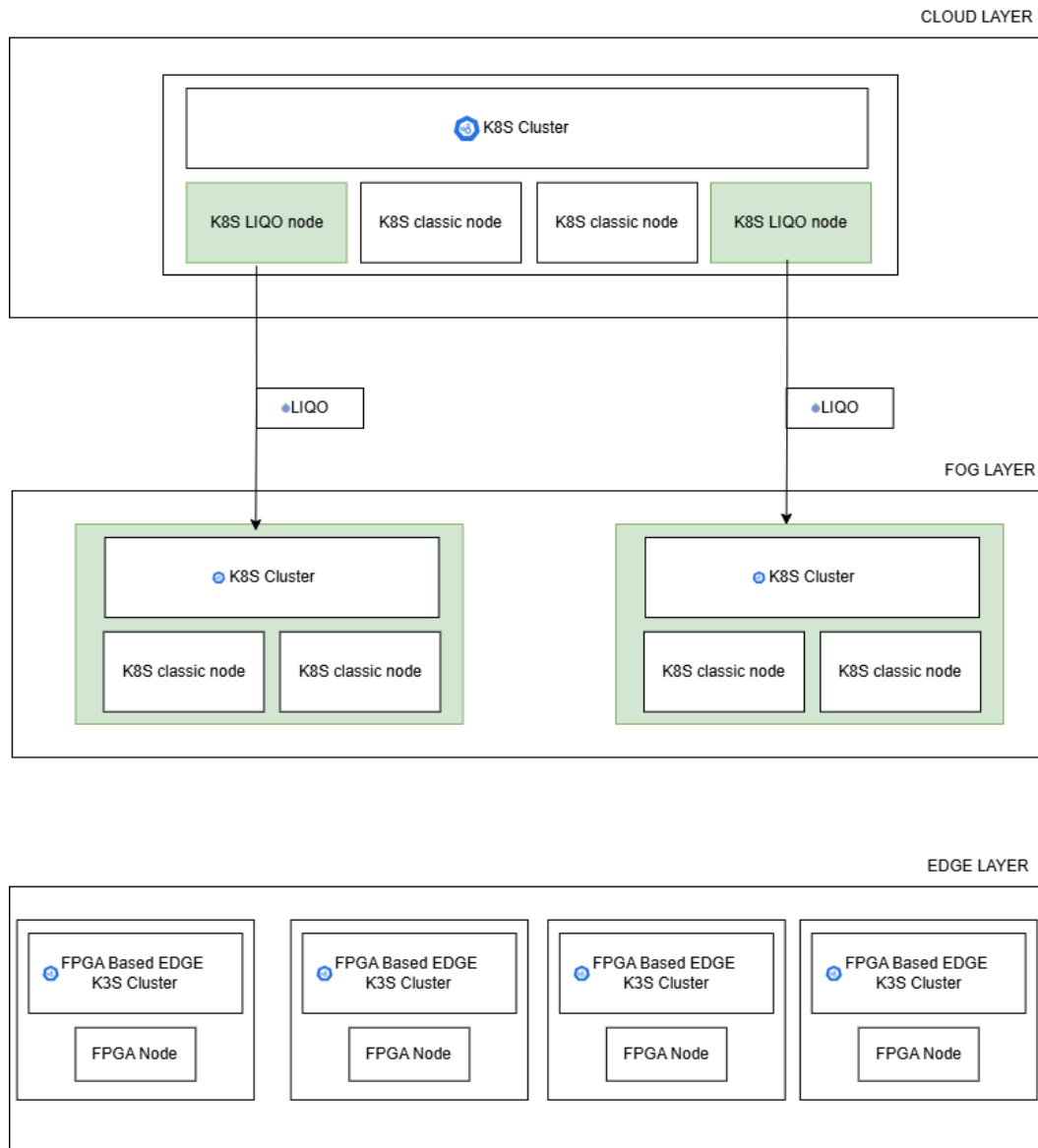


Figure 9-2: cloud to fog interaction.



Figure 9-3: edge to edge interaction.

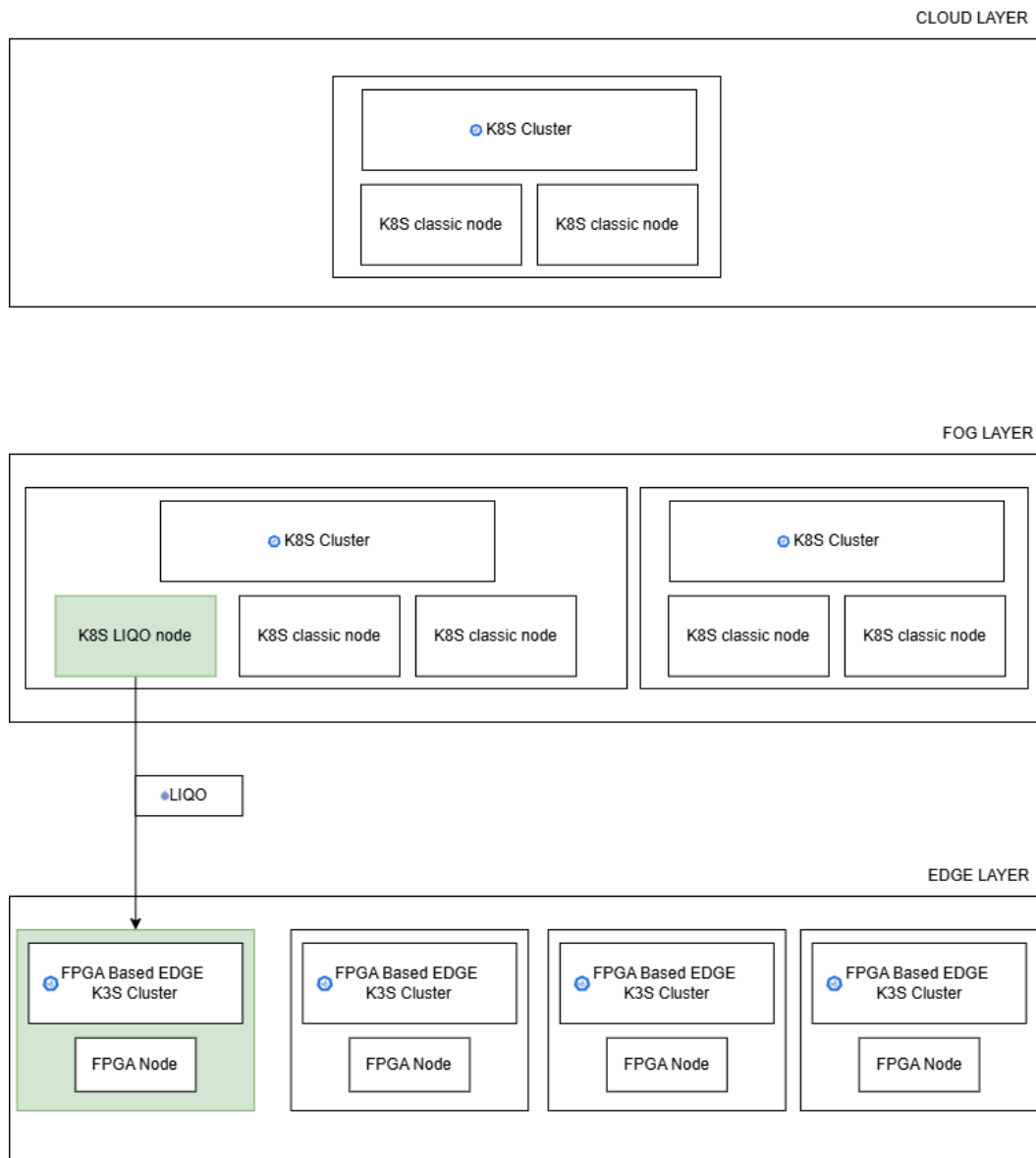


Figure 9-4: edge to fog interaction.